NYC Taxi Fare Analysis

An End-to-End Data Pipeline Using SQL, Python, and Excel
Author: Rain999-sketch
Date: 12 September 2026
GitHub: https://github.com/Rain999-sketch/nyc-taxi-analysis

1. Project Overview

1.1 Goal

The goal of this project was to analyze 18.1 million NYC taxi trips using a complete data pipeline: SQL for cleaning, Python for analysis and modeling, and Excel to create a dashboard.

1.2 Dataset

Source: NYC TLC Trip Record Data (Jan–Aug 2023)
Size: 19.4 million rows, 2.23 GB
Link:https://www.kaggle.com/datasets/nagasai524/nyc-taxi-trip-records-from-jan-2023-to-jun-2023?select=nyc_yellow_taxi_trip_records_from_Jan_to_Aug_2023.csv

1.3 Tools Used

- SQL (SQLite): Data cleaning, deduplication, filtering, export.
- Python (Pandas, NumPy, Matplotlib, Seaborn, Scikit-learn): Feature engineering, EDA, statistical testing, linear regression.
- Excel (Online): PivotTables, charts, slicers, KPI cards.

1.4 Key Results

- 5 AM fares spike to about $27 (full data; about $26 in the 200k sample). Splitting trips with a broader airport flag shows non-airport trips at 5 AM average $13.23, below their ~$14.50 all-day average, while airport trips stay near $58, so the spike is an airport mix effect: about 30.8% of 5 AM trips are airport-related vs about 10.8% at noon.
- SQL validation removed invalid records, leaving 18,169,048 rows for analysis. Extreme numerical outliers are handled separately in the Python EDA phase capped at the 99th percetile.
- Linear regression: $R^2 = 0.916$, MAE = $1.95, against a distance-only baseline of $R^2 = 0.897$, MAE = $2.80. Trip distance alone accounts for almost all explainable fare variation, consistent with the TLC's published distance-based rate card.
- The weekend flag shows no statistically significant difference ($p = 0.43$), but this hides effects between Sunday and Saturday: Saturday has the lowest average fare ($17.31), Sunday the highest ($20.77). Saturday pulls fare down and Sunday pulls up, canceling each other.

1. Project Overview **1**
1.1 Goal 2
1.2 Dataset 2
1.3 Tools Used 2
1.4 Key Results 3
2. SQL Phase: Data Cleaning **3**
2.1 Import Raw Data 3
2.2 Initial Inspection 3
2.3 Data Type Audit 4
2.4 NULL Check 5
2.5 Duplicate Check 6
2.6 Logical Consistency Check 7
2.7 Outlier Scan 7
2.8 Categorical Value Audit 8
2.9 Build the Analytical Dataset 8
2.10 Save and Export 10
3. Python Phase: Analysis & Modeling **11**
3.1 Setup & Memory Optimization 11
3.2 Data Validation 13
3.3 Feature Engineering 17
3.4 Outlier Detection & Capping 19
3.5 Exploratory Data Analysis (EDA) 21
3.6 Time Aggregations 25
3.7 Testing the 5 AM Airport Hypothesis 27
3.8 Correlation Heatmap 32
3.9 Statistical Testing 34
3.10 Linear Regression Model 35
3.11 Excel Export 43
4. Excel Phase: Interactive Dashboard **45**
4.1 Data Sample 45
4.2 Helper Columns for Slicers 46
4.3 PivotTables Created 47
4.4 Charts created 49
4.5 KPI Cards 53
4.6 Slicers 54
4.7 Layout 55
4.8 Excel Section Summary 56
5. Key Findings & Business Insights **56**
5.1 Trip Distance Drives Fare, but Revenue per Minute Is U-Shaped 57
5.2 Weekend Surcharges Are Not Viable, But Sunday Is Worth a Pilot 57
5.3 5 AM Is a Predictable High-Fare Window Driven by Airport Trips 57
5.4 Payment Type Doesn't Affect Fare, No Operational Change Needed 58
5.5 Tip Analysis Only Reflects Card Payments 58
5.6 Summary of Operational Recommendations 58

6. Limitations & Future Improvements **58**
6.1 Limitations 58
6.2 Possible Improvements 59
7. Appendix **59**
7.1 SQL Code 59
7.2 Python Code 62

2. SQL Phase: Data Cleaning

2.1 Import Raw Data

SQLite is a lightweight, file-based database engine. It requires no server setup and is ideal for local data cleaning tasks, which is why we picked it.

Input:

```
.mode csv
.import nyc_yellow_taxi_trip_records_from_Jan_to_Aug_2023.csv taxi_trips
```

We used SQLite's .import command to load the 2.23 GB CSV into a table called taxi_trips.
.mode csv changes the output and input formatting to Comma-Separated Values, CSV for short
.import is an SQL command that loads CSV into the table, and it treats every value as text by default

Result: This created a staging table with all data stored as text.

2.2 Initial Inspection

We ran basic checks to understand the data.

Input:

```
SELECT COUNT(*) FROM taxi_trips;
SELECT * FROM taxi_trips LIMIT 5;
```

Here on the 1st line we count all rows including NULL ones, on the 2nd line we check the first 5 rows of every column.

Output:

COUNT(*)
19493059

```
sqlite> SELECT * FROM taxi_trips LIMIT 5;
```

?	VendorID	tpep_pickup_datetime	tpep_dropoff_date...	passenger_count	trip_distance	RatecodeID	store_and_fwd_flag	PULocationID	DOLocationID	payment_type
'0'	'2'	2023-06-30 23:59:59	2023-07-01 00:47:49	'2.0'	'17.62'	'2.0'	N	'132'	'230'	'1'
'1'	'2'	2023-06-30 23:59:57	2023-07-01 00:17:36	'1.0'	'3.32'	'1.0'	N	'255'	'198'	'1'
'2'	'1'	2023-06-30 23:59:55	2023-07-01 00:14:20	'1.0'	'2.8'	'1.0'	N	'231'	'107'	'1'
'3'	'2'	2023-06-30 23:59:55	2023-07-01 00:05:52	'1.0'	'0.89'	'1.0'	N	'237'	'163'	'1'
'4'	'2'	2023-06-30 23:59:55	2023-07-01 00:07:08	'4.0'	'1.56'	'1.0'	N	'211'	'90'	'1'

fare_amount	extra	mta_tax	tip_amount	tolls_amount	improvement_surch...	total_amount	congestion_surcharge	Airport_fee_20	airport_fee_21
'70.0'	'0.0'	'0.5'	'0.0'	'6.55'	'1.0'	'82.3'	'2.5'	'1.75'	
'18.4'	'1.0'	'0.5'	'2.5'	'0.0'	'1.0'	'23.4'	'0.0'	'0.0'	
'14.9'	'3.5'	'0.5'	'3.95'	'0.0'	'1.0'	'23.85'	'2.5'	'0.0'	
'7.2'	'1.0'	'0.5'	'2.0'	'0.0'	'1.0'	'14.2'	'2.5'	'0.0'	
'10.0'	'1.0'	'0.5'	'3.0'	'0.0'	'1.0'	'18.0'	'2.5'	'0.0'	

Result: 19,493,059 rows. The first row was a header, which we deleted.

2.3 Data Type Audit

We checked how SQLite stored each column.

Input:

```
PRAGMA table_info(taxi_trips);
SELECT typeof(VendorID), typeof(fare_amount) FROM taxi_trips LIMIT 1;
```

PRAGMA is used to check information about the structure of the table. The first column is named "?", this is SQLite's placeholder for an unnamed column created during ".import". It contains row IDs but was excluded from the final cleaned table.

Output:

```
sqlite> PRAGMA table_info(taxi_trips);
```

cid	name	type	notnull	dflt_value	pk
0	?		0	NULL	0
1	VendorID		0	NULL	0
2	tpep_pickup_datetime		0	NULL	0
3	tpep_dropoff_datetime		0	NULL	0
4	passenger_count		0	NULL	0
5	trip_distance		0	NULL	0
6	RatecodeID		0	NULL	0
7	store_and_fwd_flag		0	NULL	0
8	PULocationID		0	NULL	0
9	DOLocationID		0	NULL	0
10	payment_type		0	NULL	0
11	fare_amount		0	NULL	0
12	extra		0	NULL	0
13	mta_tax		0	NULL	0
14	tip_amount		0	NULL	0
15	tolls_amount		0	NULL	0
16	improvement_surcharge		0	NULL	0
17	total_amount		0	NULL	0
18	congestion_surcharge		0	NULL	0
19	Airport_fee_20		0	NULL	0
20	airport_fee_21		0	NULL	0

- cid = Our column index
- name= it's just the name of the column
- type = the declared data type the column is supposed to hold. Right now it's empty, meaning SQLite enforces no type, so it can store any value, which can bring trouble for later.
- notnull = if it's 0 it means it allows NULL, if it's 1 it doesn't
- dflt_value = this is the default value if there is nothing
- pk = if this is 1 it means there is a primary key, if it's 0 it means the column is not a primary key

Result: All columns were stored as TEXT. We needed to CAST them to INTEGER or REAL.

2.4 NULL Check

We need to check if we have missing values, so we check any Nulls in VendorID, fare_amount and taxi_trips.

Input:

```
SELECT COUNT(*), COUNT(VendorID), COUNT(fare_amount) FROM taxi_trips;
```

Output:

```
sqlite> SELECT COUNT(*), COUNT(VendorID), COUNT(fare_amount) FROM taxi_trips;
```

COUNT(*)	COUNT(VendorID)	COUNT(fare_amount)
19493059	19493059	19493059

Result: No NULLs found.

2.5 Duplicate Check

We looked for duplicate trips using a natural key (pickup, dropoff, fare, distance).

Input:

```
SELECT tpep_pickup_datetime, tpep_dropoff_datetime, fare_amount,
trip_distance, COUNT(*)
FROM taxi_trips
GROUP BY tpep_pickup_datetime, tpep_dropoff_datetime, fare_amount,
trip_distance
HAVING COUNT(*) > 1 LIMIT 10;
```

Our first line picks the columns we want to use to check if there are duplicates, we then use count to see how many rows have any of these columns share the combinations of these columns, and then we group based off the columns, at the end we run a command to let us see only 10 of these dupes, this way it won't crash by showing all of them.

Output:

```
sqlite> SELECT tpep_pickup_datetime, tpep_dropoff_datetime, fare_amount, trip_distance, COUNT(*)
   ...> FROM taxi_trips
   ...> GROUP BY tpep_pickup_datetime, tpep_dropoff_datetime, fare_amount, trip_distance
   ...> HAVING COUNT(*) > 1
   ...> LIMIT 10;
```

tpep_pickup_datetime	tpep_dropoff_date...	fare_amount	trip_distance	COUNT(*)
2023-01-02 04:23:28	2023-01-02 04:23:58	'0.0'	'0.02'	2
2023-01-02 05:12:19	2023-01-02 05:41:45	'0.0'	'17.07'	2
2023-01-02 06:32:52	2023-01-02 07:05:44	'0.0'	'16.71'	2
2023-01-02 07:29:44	2023-01-02 07:59:18	'0.0'	'1.11'	2
2023-01-02 11:03:12	2023-01-02 11:05:47	'0.0'	'0.01'	2
2023-01-03 08:27:38	2023-01-03 08:59:16	'0.0'	'8.34'	2
2023-01-03 09:13:08	2023-01-03 09:36:10	'0.0'	'8.09'	2
2023-01-03 09:32:18	2023-01-03 09:32:53	'0.0'	'0.14'	2
2023-01-03 12:15:16	2023-01-03 12:19:05	'0.0'	'0.0'	2
2023-01-03 13:57:52	2023-01-03 14:23:09	'0.0'	'3.24'	2

Note on deduplication: We identified 1,016 duplicate groups on the natural key (pickup, dropoff, fare, distance). However, we chose not to dedupe on that key. Two rows with identical natural keys but different VendorIDs or payment_types could represent legitimately distinct trips.

Result: 1,016 duplicate groups (2,032 rows), mostly with fare_amount = 0.

2.6 Logical Consistency Check

We check for impossible trips (dropoff before pickup).

Input:

```
SELECT COUNT(*) FROM taxi_trips WHERE tpep_dropoff_datetime <=
tpep_pickup_datetime;
```

Our 1st line counts all the rows where somehow the drop off time is lower than the pick up time.

Output:

```
sqlite> SELECT COUNT(*) FROM taxi_trips WHERE tpep_dropoff_datetime <= tpep_pickup_datetime;
```

COUNT(*)
7434

Result: 7,434 bad rows.

2.7 Outlier Scan

We now search for extreme values, we check the min and max for fare_amount, trip_distance and passenger_count

Input:

```
SELECT
    MAX(CAST(fare_amount AS REAL)) AS max_fare,
    MIN(CAST(fare_amount AS REAL)) AS min_fare,
    MAX(CAST(trip_distance AS REAL)) AS max_distance,
    MIN(CAST(trip_distance AS REAL)) AS min_distance,
    MAX(CAST(passenger_count AS INTEGER)) AS max_passengers,
    MIN(CAST(passenger_count AS INTEGER)) AS min_passengers
FROM taxi_trips;
```

Output:

```
sqlite> SELECT
   ...>     MAX(CAST(fare_amount AS REAL)) AS max_fare,
   ...>     MIN(CAST(fare_amount AS REAL)) AS min_fare,
   ...>     MAX(CAST(trip_distance AS REAL)) AS max_distance,
   ...>     MIN(CAST(trip_distance AS REAL)) AS min_distance,
   ...>     MAX(CAST(passenger_count AS INTEGER)) AS max_passengers,
   ...>     MIN(CAST(passenger_count AS INTEGER)) AS min_passengers
   ...> FROM taxi_trips;
```

max_fare	min_fare	max_distance	min_distance	max_passengers	min_passengers
386983.63	-959.9	342344.85	0.0	9	0

Our first pass at the outlier scan used MAX() directly on text columns. SQLite compared strings lexicographically, so '999.0' appeared as the maximum fare instead of the true numeric max. We caught this when our Python analysis reported a $386,983 maximum, a value that couldn't exist as text. We reran the query with CAST(... AS REAL) and confirmed the real range. This taught us to always verify data types before applying aggregate functions.

Result:

- fare_amount: min = -959.9 (negative fare, impossible), max = 386983.63 (too big)
- trip_distance: min = 0.0 (zero-distance trips), max = 342344.85 miles (physically impossible)
- passenger_count: min = 0 (fare for the driver?), max = 9.0 (exceeds NYC taxi capacity of 6)

These extreme values confirmed the need for filters in the final cleaning step.

2.8 Categorical Value Audit

We checked the distinct values in two key categorical columns to identify invalid entries before cleaning and see what values they can take.

Input:

```
SELECT DISTINCT RatecodeID FROM taxi_trips;
SELECT DISTINCT store_and_fwd_flag FROM taxi_trips;
```

Output:



Result: The first query returned valid codes (1-6) but also invalid values: 99.0 and an empty string (''). The second query returned 'Y', 'N', and an empty string.

2.9 Build the Analytical Dataset

We combined everything into one CREATE TABLE AS SELECT command. CAST is used to give the data type we desire for the column, the ones that don't need change like pick up time are left unchanged, and we make sure that this clean table won't have impossible values using WHERE, UPPER just makes converts it into capital letters, while TRIM removes empty spaces.

Input:

```
CREATE TABLE taxi_trips_final AS
SELECT DISTINCT
    CAST(VendorID AS INTEGER) AS VendorID,
    tpep_pickup_datetime,
    tpep_dropoff_datetime,
    CAST(passenger_count AS INTEGER) AS passenger_count,
    CAST(trip_distance AS REAL) AS trip_distance,
    CAST(RatecodeID AS INTEGER) AS RatecodeID,
    UPPER(TRIM(store_and_fwd_flag)) AS store_and_fwd_flag,
    CAST(PULocationID AS INTEGER) AS PULocationID,
    CAST(DOLocationID AS INTEGER) AS DOLocationID,
    CAST(payment_type AS INTEGER) AS payment_type,
    CAST(fare_amount AS REAL) AS fare_amount,
    CAST(extra AS REAL) AS extra,
```

```
    CAST(mta_tax AS REAL) AS mta_tax,
    CAST(tip_amount AS REAL) AS tip_amount,
    CAST(tolls_amount AS REAL) AS tolls_amount,
    CAST(improvement_surcharge AS REAL) AS improvement_surcharge,
    CAST(total_amount AS REAL) AS total_amount,
    CAST(congestion_surcharge AS REAL) AS congestion_surcharge,
    CAST(Airport_fee_20 AS REAL) AS Airport_fee_20,
    CAST(airport_fee_21 AS REAL) AS airport_fee_21
FROM taxi_trips
WHERE CAST(fare_amount AS REAL) > 0
  AND CAST(trip_distance AS REAL) > 0
  AND CAST(trip_distance AS REAL) < 1000
  AND CAST(passenger_count AS INTEGER) > 0
  AND CAST(passenger_count AS INTEGER) <= 6
  AND tpep_dropoff_datetime > tpep_pickup_datetime
  AND CAST(RatecodeID AS INTEGER) IN (1, 2, 3, 4, 5, 6)
  AND UPPER(TRIM(store_and_fwd_flag)) IN ('Y', 'N');
```

The CREATE TABLE command itself doesn't produce output, it creates the new table silently. We verified the result with the following queries.

```
SELECT COUNT (*) FROM taxi_trips_final;
PRAGMA table_info(taxi_trips_final);
SELECT typeof(VendorID), typeof(fare_amount)  FROM taxi_trips_final LIMIT
1;
```

We ran PRAGMA table_info to confirm the
schema. The numeric columns now show INT or REAL types (VendorID, fare_amount, trip_distance, etc.), while datetime and flag columns remain untyped, which is correct because SQLite stores dates and text as text.

We also confirmed with typeof():

SELECT typeof(VendorID), typeof(fare_amount) FROM taxi_trips_final LIMIT 1;
Result: 'integer' and 'real'. This proves the CAST() statements worked.

Output:

```
sqlite> SELECT COUNT(*) FROM taxi_trips_final;
┌──────────┐
│ COUNT(*) │
├──────────┤
│ 18169048 │
└──────────┘
```

```
sqlite> PRAGMA table_info(taxi_trips_final);
```

cid	name	type	notnull	dflt_value	pk
0	VendorID	INT	0	NULL	0
1	tpep_pickup_datetime		0	NULL	0
2	tpep_dropoff_datetime		0	NULL	0
3	passenger_count	INT	0	NULL	0
4	trip_distance	REAL	0	NULL	0
5	RatecodeID	INT	0	NULL	0
6	store_and_fwd_flag		0	NULL	0
7	PULocationID	INT	0	NULL	0
8	DOLocationID	INT	0	NULL	0
9	payment_type	INT	0	NULL	0
10	fare_amount	REAL	0	NULL	0
11	extra	REAL	0	NULL	0
12	mta_tax	REAL	0	NULL	0
13	tip_amount	REAL	0	NULL	0
14	tolls_amount	REAL	0	NULL	0
15	improvement_surcharge	REAL	0	NULL	0
16	total_amount	REAL	0	NULL	0
17	congestion_surcharge	REAL	0	NULL	0
18	Airport_fee_20	REAL	0	NULL	0
19	airport_fee_21	REAL	0	NULL	0

```
sqlite> SELECT typeof(VendorID), typeof(fare_amount) FROM taxi_trips_final LIMIT 1;
```

typeof(VendorID)	typeof(fare_amount)
integer	real

What it did:
- CAST all columns to correct types.
- Standardized store_and_fwd_flag with UPPER(TRIM()).
- Filtered out bad rows (negative fares, zero distance, invalid passengers, bad timestamps, invalid rate codes).
- Removed exact duplicates with SELECT DISTINCT.

Result: 18,169,048 clean rows (down from 19.4 million).

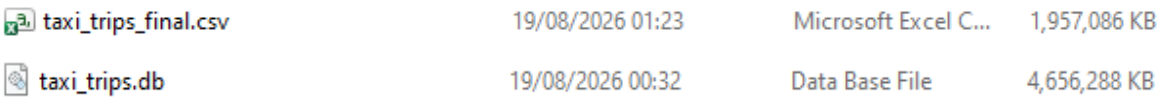
2.10 Save and Export

We saved the database and exported the clean table to CSV.

Input:

```
.save taxi_trips.db
.mode csv
.headers on
.output taxi_trips_final.csv
SELECT * FROM taxi_trips_final;
.output stdout
```

Output:



Result: taxi_trips_final.csv ready for Python and Excel.

Metric	Value
Raw rows imported	19,493,059
Clean rows exported	18,169,048
Rows removed	1,324,011 (6.8%)
Duplicate groups found	1,016 (2,032 rows)
Impossible timestamps	7,434

3. Python Phase: Analysis & Modeling

3.1 Setup & Memory Optimization

We loaded the cleaned CSV with explicitly defined data types to avoid running out of memory.

Input:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sea
import warnings
warnings.filterwarnings('ignore')
from scipy import stats
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, r2_score

# Memory optimization
dtype_dict = {
    'VendorID': 'int8',
    'passenger_count': 'int8',
    'trip_distance': 'float32',
    'RatecodeID': 'int8',
    'PULocationID': 'int16',
    'DOLocationID': 'int16',
    'payment_type': 'int8',
    'fare_amount': 'float32',
    'extra': 'float32',
    'mta_tax': 'float32',
    'tip_amount': 'float32',
    'tolls_amount': 'float32',
    'improvement_surcharge': 'float32',
    'total_amount': 'float32',
    'congestion_surcharge': 'float32',
    'Airport_fee_20': 'float32',
    'airport_fee_21': 'float32'
}
```

```
# Loads the data
sw = pd.read_csv(
    'taxi_trips_final.csv',
    dtype=dtype_dict,
    parse_dates=['tpep_pickup_datetime', 'tpep_dropoff_datetime']
)
```

Output:

```
Rows loaded: 18169048
Columns: 20
Memory used: 1399.02 MB
```

Results:

Pandas defaults to int64 (8 bytes) for integers and float64 (8 bytes) for decimals. For 18 million rows and 18 columns, that would require roughly 2.6 GB of RAM just for the data, enough to crash or severely slow down most laptops. This cut memory usage by roughly 45%, from an estimated 2.5 GB down to 1.4 GB.

Examples of type selection:

- passenger_count => int8 (1 byte). Max value is 6, so it fits comfortably.
- PULocationID => int16 (2 bytes). Location IDs go up to approximately 265, which exceeds the int8 limit of 127.
- fare_amount => float32 (4 bytes). Decimals with enough precision for dollars and cents.

3.2 Data Validation

We checked the cleaned data for five things: duplicates, logical consistency, valid categorical values, airport fee mutual exclusivity, and fare-component consistency.

Input:

```
# Checking the data
print(f"Duplicate rows: {sw.duplicated().sum()}")
print("Rows loaded:", sw.shape[0])
print("Columns:", sw.shape[1])
print("Memory used:", round(sw.memory_usage(deep=True).sum() / 1e6, 2),
"MB")
print(sw.head())
print("\n---- DATA INFO ----")
sw.info()

print("\n---- DATA VALIDATION ----")

# Checking if the dropoff is always after pickup
```

```
bad_times = (sw['tpep_dropoff_datetime'] <=
sw['tpep_pickup_datetime']).sum()
print(f"Rows with dropoff <= pickup: {bad_times}")

# Checking airport fee columns
both_nonzero = ((sw['Airport_fee_20'] > 0) & (sw['airport_fee_21'] >
0)).sum()
print(f"Rows with both airport fees non-zero: {both_nonzero}")
print(f"Rows with Airport_fee_20 > 0: {(sw['Airport_fee_20'] > 0).sum()}")
print(f"Rows with airport_fee_21 > 0: {(sw['airport_fee_21'] > 0).sum()}")

# Checking unique values in categorical columns
for col in ['VendorID', 'RatecodeID', 'payment_type',
'store_and_fwd_flag']:
    print(f"{col}: {sw[col].unique()}")

# Comparing fare components sum to total_amount
sw['calc_total'] = (
    sw['fare_amount'] + sw['extra'] + sw['mta_tax'] +
    sw['tip_amount'] + sw['tolls_amount'] +
    sw['improvement_surcharge'] + sw['congestion_surcharge'] +
    sw['Airport_fee_20'] + sw['airport_fee_21']
)
mismatch = (sw['calc_total'] - sw['total_amount']).abs() > 0.01
print(f"Rows where components don't sum to total: {mismatch.sum()}
({mismatch.mean()*100:.2f}%)")


print(f"Fraction with non-zero congestion
surcharge:{(sw['congestion_surcharge'] != 0).mean():.4f}")
diff = sw['calc_total'] - sw['total_amount']
mismatched = sw[~np.isclose(sw['calc_total'], sw['total_amount'],
atol=0.01)]
matches_surcharge = np.isclose(
    diff[mismatched.index],
    sw.loc[mismatched.index, 'congestion_surcharge'],
    atol=0.01
)
```

```
print(f"Fraction of mismatched rows where gap = congestion surcharge:
{matches_surcharge.mean()*100:.2f}%")

sw['mismatch'] = ~np.isclose(sw['calc_total'], sw['total_amount'],
atol=0.01)
print("\nMismatch rate by vendor:")
print(sw.groupby('VendorID')['mismatch'].mean())



failing = sw[mismatch].head(3)
print(failing[['fare_amount', 'extra', 'mta_tax', 'tip_amount',
               'tolls_amount', 'improvement_surcharge',
'congestion_surcharge',
               'Airport_fee_20', 'airport_fee_21', 'calc_total',
'total_amount']])
```

Output:

```
Duplicate rows: 0
```

	VendorID	tpep_pickup_datetime	tpep_dropoff_datetime	passenger_count	trip_distance	RatecodeID	...
0	2	2023-06-30 23:59:59	2023-07-01 00:47:49	2	17.620001	2	...
1	2	2023-06-30 23:59:57	2023-07-01 00:17:36	1	3.320000	1	...
2	1	2023-06-30 23:59:55	2023-07-01 00:14:20	1	2.800000	1	...
3	2	2023-06-30 23:59:55	2023-07-01 00:05:52	1	0.890000	1	...
4	2	2023-06-30 23:59:55	2023-07-01 00:07:08	4	1.560000	1	...

[5 rows x 20 columns]

...	tolls_amount	improvement_surcharge	total_amount	congestion_surcharge	Airport_fee_20	airport_fee_21
...	6.55	1.0	82.300003	2.5	1.75	0.0
...	0.00	1.0	23.400000	0.0	0.00	0.0
...	0.00	1.0	23.850000	2.5	0.00	0.0
...	0.00	1.0	14.200000	2.5	0.00	0.0
...	0.00	1.0	18.000000	2.5	0.00	0.0

```
---- DATA INFO ----
<class 'pandas.DataFrame'>
RangeIndex: 18169048 entries, 0 to 18169047
Data columns (total 20 columns):
 #   Column                 Dtype
---  ------                 -----
 0   VendorID               int8
 1   tpep_pickup_datetime   datetime64[us]
 2   tpep_dropoff_datetime  datetime64[us]
 3   passenger_count        int8
 4   trip_distance          float32
 5   RatecodeID             int8
 6   store_and_fwd_flag     str
 7   PULocationID           int16
 8   DOLocationID           int16
 9   payment_type           int8
 10  fare_amount            float32
 11  extra                  float32
 12  mta_tax                float32
 13  tip_amount             float32
 14  tolls_amount           float32
 15  improvement_surcharge  float32
 16  total_amount           float32
 17  congestion_surcharge   float32
 18  Airport_fee_20         float32
 19  airport_fee_21         float32
dtypes: datetime64[us](2), float32(11), int16(2), int8(4), str(1)
memory usage: 1.3 GB
```

```
---- DATA VALIDATION ----
Rows with dropoff <= pickup: 0
Rows with both airport fees non-zero: 0
Rows with Airport_fee_20 > 0: 1305009
Rows with airport_fee_21 > 0: 254608
VendorID: [2 1]
RatecodeID: [2 1 4 5 3 6]
payment_type: [1 4 2 3]
store_and_fwd_flag: <ArrowStringArray>
['N', 'Y']
Length: 2, dtype: str
Rows where components don't sum to total: 4514225 (24.85%)
Fraction with non-zero congestion surcharge:0.9343
Fraction of mismatched rows where gap = congestion surcharge: 92.30%
```

```
Mismatch rate by vendor:
VendorID
1    0.975302
2    0.000481
Name: mismatch, dtype: float64
    fare_amount  extra  mta_tax  tip_amount  tolls_amount  improvement_surcharge
2          14.9    3.5      0.5        3.95           0.0                    1.0
8          14.9    3.5      0.5        4.00           0.0                    1.0
10          5.8    3.5      0.5        2.15           0.0                    1.0
```

```
congestion_surcharge  Airport_fee_20  airport_fee_21  calc_total  total_amount
                 2.5             0.0             0.0   26.350000         23.85
                 2.5             0.0             0.0   26.400000         23.90
                 2.5             0.0             0.0   15.450001         12.95
```

Result:
- Duplicate rows: 0
- Rows with dropoff <= pickup: 0
- Rows with both airport fees non-zero: 0 (confirms mutual exclusivity)
- Airport_fee_20 applied to 1,305,009 trips; airport_fee_21 applied to 254,608 trips, these are distinct fee schedules from different periods
- All categorical columns show only expected values

Note on Airport_fee columns: The Kaggle CSV contained two columns originally both named "Airport_fee". On export, they were renamed with position suffixes. They represent two TLC fee schedules and are mutually exclusive, our check confirmed no row has both non-zero.

While validating the fare-component sum, we found a 24.85% mismatch.Investigating further, the gap was exactly $2.50, the congestion surcharge, in 92.3% of mismatched rows, and it split 97.5% / 0.05% by VendorID. Vendor 1's meters exclude the surcharge from total_amount; Vendor 2's include it. We documented this as a systematic vendor-system difference rather than a data error.

3.3 Feature Engineering

We created 9 new features plus 3 binned versions for visualization (12 total) . These fall into three groups:

1) Trip metrics, trip_duration_min, avg_speed_mph, fare_per_mile
2) Time features, pickup_hour, pickup_day_of_week, pickup_month
3) Behavioral flags, total_surcharges, tip_pct, is_weekend

Input:

```
print("\n---- FEATURE ENGINEERING ----")

# 1) Core trip metrics
sw['trip_duration_min'] = (sw['tpep_dropoff_datetime'] -
sw['tpep_pickup_datetime']).dt.total_seconds() / 60
sw['avg_speed_mph'] = sw['trip_distance'] / (sw['trip_duration_min'] / 60)
sw['fare_per_mile'] = sw['fare_amount'] / sw['trip_distance']

# 2) Time features
sw['pickup_hour'] = sw['tpep_pickup_datetime'].dt.hour
sw['pickup_day_of_week'] = sw['tpep_pickup_datetime'].dt.dayofweek  #
0=Mon, 6=Sun
```

```
sw['pickup_month'] = sw['tpep_pickup_datetime'].dt.month

# 3) Total surcharges
sw['total_surcharges'] = (
    sw['extra'] + sw['mta_tax'] + sw['tolls_amount'] +
    sw['improvement_surcharge'] + sw['congestion_surcharge'] +
    sw['Airport_fee_20'] + sw['airport_fee_21']
).fillna(0)

# Tip percentage
sw['tip_pct'] = sw['tip_amount'] / sw['fare_amount']

# Weekend flag
sw['is_weekend'] = (sw['pickup_day_of_week'] >= 5).astype('int8')

# 4) Handles division by 0
sw['avg_speed_mph'] = sw['avg_speed_mph'].replace([np.inf, -np.inf],
np.nan).fillna(0)
sw['fare_per_mile'] = sw['fare_per_mile'].replace([np.inf, -np.inf],
np.nan).fillna(0)
sw['tip_pct'] = sw['tip_pct'].replace([np.inf, -np.inf], np.nan).fillna(0)

# 5) Optimizing memory for the new columns
sw['total_surcharges'] = sw['total_surcharges'].astype('float32')
sw['tip_pct'] = sw['tip_pct'].astype('float32')

# 6. Final summary
print(f"New shape: {sw.shape[0]} rows, {sw.shape[1]} columns")
print("\nNew features added (12 total, including bins):")
print("  - distance_bin, fare_bin, duration_bin (for visualizations)")
print("  - trip_duration_min")
print("  - avg_speed_mph")
print("  - fare_per_mile")
print("  - pickup_hour, pickup_day_of_week, pickup_month")
print("  - total_surcharges")
print("  - tip_pct")
print("  - is_weekend")
print("\nSample of the new features:")
```

```
print(sw[['trip_duration_min', 'avg_speed_mph', 'fare_per_mile',
'total_surcharges', 'tip_pct', 'is_weekend']].head())
```

Output:

```
---- FEATURE ENGINEERING ----
New shape: 18169048 rows, 31 columns

New features added (12 total, including bins):
  - distance_bin, fare_bin, duration_bin (for visualizations)
  - trip_duration_min
  - avg_speed_mph
  - fare_per_mile
  - pickup_hour, pickup_day_of_week, pickup_month
  - total_surcharges
  - tip_pct
  - is_weekend

Sample of the new features:
   trip_duration_min  avg_speed_mph  fare_per_mile  total_surcharges   tip_pct  is_weekend
0          47.833333      22.101743       3.972758              12.3  0.000000           0
1          17.650000      11.286119       5.542169               2.5  0.135870           0
2          14.416667      11.653179       5.321428               7.5  0.265101           0
3           5.950000       8.974790       8.089888               5.0  0.277778           0
4           7.216667      12.969976       6.410257               5.0  0.300000           0
```

Result: Raw data rarely has the features you need for analysis.
Creating trip_duration_min lets us analyze time-based pricing; tip_pct
reveals customer behavior; is_weekend enables demand pattern comparisons.

3.4 Outlier Detection & Capping

We used percentiles (1st and 99th) to identify outliers, then capped them for EDA.

Input:

```
print("\n---- OUTLIER DETECTION ----")

# 1) Stats with percentiles
numeric_cols = [
    'trip_distance',
    'fare_amount',
    'trip_duration_min',
    'avg_speed_mph',
    'fare_per_mile',
    'total_surcharges',
    'tip_pct'
]
```

```
print(sw[numeric_cols].describe(percentiles=[0.01, 0.25, 0.5, 0.75,
0.99]).T)

# 2) Threshold counts
thresholds = {
    'trip_distance': 100,
    'fare_amount': 150,
    'trip_duration_min': 180,
    'avg_speed_mph': 80,
    'fare_per_mile': 20,
    'tip_pct': 0.5
}

print("\n---- OUTLIER COUNTS BY THRESHOLD ----")
for col, threshold in thresholds.items():
    n = (sw[col] > threshold).sum()
    pct = (n/len(sw))* 100
    print(f"{col}: {n} rows above {threshold} ({pct:.2f}%)")

print("\n---- CREATING CLEANED DATASET ----")
sw_eda = sw.sample(200000, random_state=42).copy()
sw_eda['trip_distance'] = sw_eda['trip_distance'].clip(upper=100)
sw_eda['fare_amount'] = sw_eda['fare_amount'].clip(upper=150)
sw_eda['trip_duration_min'] = sw_eda['trip_duration_min'].clip(upper=180)
sw_eda['avg_speed_mph'] = sw_eda['avg_speed_mph'].clip(upper=80)
sw_eda['fare_per_mile'] = sw_eda['fare_per_mile'].clip(upper=20)
sw_eda['tip_pct'] = sw_eda['tip_pct'].clip(upper=0.5)
print(f"Cleaned dataset shape: {sw_eda.shape}")

sw_eda_sample = sw_eda.sample(10000, random_state=42)
sw_eda_sample.to_csv('taxi_trips_eda_sample_small.csv', index=False)
```

Output:

```
---- OUTLIER DETECTION ----
                     count       mean         std       min        1%       25%       50%       75%       99%          max
trip_distance     18169048.0   3.476842    4.545691  0.010000  0.300000  1.100000  1.800000  3.400000  20.209999    837.500000
fare_amount       18169048.0  19.300184   92.511482  0.010000  4.400000  9.300000 13.500000 21.200001  73.000000 386983.625000
trip_duration_min 18169048.0  16.969464   43.309630  0.016667  1.983333  7.533333 12.233333 19.683333  64.850000   7053.616667
avg_speed_mph     18169048.0  12.395587  150.168001  0.000231  3.075511  7.488856  9.924051 13.656774  36.071147 544752.026367
fare_per_mile     18169048.0  10.740881  140.821335  0.000144  3.626943  5.601374  7.066667  8.777778  19.814814 257989.078125
total_surcharges  18169048.0   6.173171    3.261062  0.000000  1.500000  4.000000  5.000000  6.500000  19.299999    674.559998
tip_pct           18169048.0   0.210072    6.554672  0.000000  0.000000  0.093458  0.241848  0.293023   0.506897  15700.000000
```

```
---- OUTLIER COUNTS BY THRESHOLD ----
trip_distance: 139 rows above 100 (0.00%)
fare_amount: 14249 rows above 150 (0.08%)
trip_duration_min: 18611 rows above 180 (0.10%)
avg_speed_mph: 7582 rows above 80 (0.04%)
fare_per_mile: 175501 rows above 20 (0.97%)
tip_pct: 186899 rows above 0.5 (1.03%)
```

```
---- CREATING CLEANED DATASET ----
Cleaned dataset shape: (200000, 31)
```

Result: A single $10,000 fare would skew averages and break the regression model. The threshold values were chosen based on the 99th percentile: 99% of fares are under $73, so capping at $150 removes only extreme outliers without affecting the bulk of the data.

Caveat: The SQL filter removed negative fares but did not cap the upper bound. As a result, extreme values like $386,983 remained in the raw cleaned dataset. We capped these for EDA at the 99th percentile ($150), but the uncapped values persist in the underlying data.

3.5 Exploratory Data Analysis (EDA)

We created binned bar charts for:
- Trip Distance Distribution
- Fare Amount Distribution
- Trip Duration Distribution

Raw histograms with 50+ bins were hard to read because the data is heavily right-skewed. Binning groups values into meaningful ranges (e.g., 0-1 mile, 1-2 miles) so patterns are visible at a glance.

Input:

```
print("\n---- BINNED DISTRIBUTIONS ----")

# Trip distance
distance_bins = [0, 1, 2, 3, 5, 10, 20, 30, 100]
distance_labels = ['0-1', '1-2', '2-3', '3-5', '5-10', '10-20', '20-30',
'30+']
sw_eda['distance_bin'] = pd.cut(sw_eda['trip_distance'],
bins=distance_bins, labels=distance_labels)

# Fare amount
fare_bins = [0, 5, 10, 15, 20, 30, 50, 150]
fare_labels = ['0-5', '5-10', '10-15', '15-20', '20-30', '30-50', '50+']
sw_eda['fare_bin'] = pd.cut(sw_eda['fare_amount'], bins=fare_bins,
labels=fare_labels)

# Trip duration
duration_bins = [0, 5, 10, 15, 20, 30, 60, 180]
duration_labels = ['0-5', '5-10', '10-15', '15-20', '20-30', '30-60',
'60+']
sw_eda['duration_bin'] = pd.cut(sw_eda['trip_duration_min'],
bins=duration_bins, labels=duration_labels)

# 1) Trip distance distribution
plt.figure(figsize=(10, 6))
sw_eda['distance_bin'].value_counts().sort_index().plot(kind='bar',
color='steelblue', edgecolor='black')
plt.title('Trip Distance Distribution (Binned)')
plt.xlabel('Distance (miles)')
plt.ylabel('Number of Trips')
```

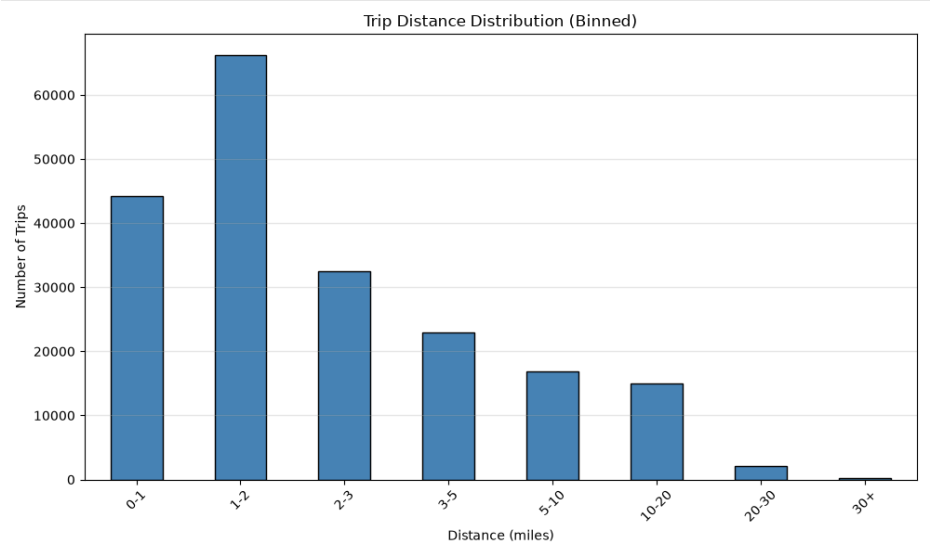
```
plt.xticks(rotation=45)
plt.grid(axis='y', alpha=0.3)
plt.tight_layout()
plt.show()
plt.close()

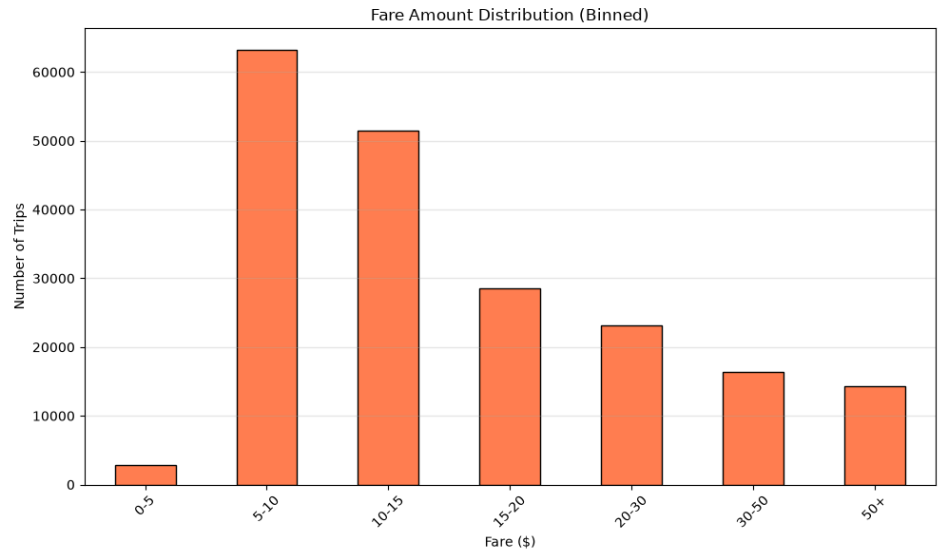
# 2) Fare amount distribution
plt.figure(figsize=(10, 6))
sw_eda['fare_bin'].value_counts().sort_index().plot(kind='bar',
color='coral', edgecolor='black')
plt.title('Fare Amount Distribution (Binned)')
plt.xlabel('Fare ($)')
plt.ylabel('Number of Trips')
plt.xticks(rotation=45)
plt.grid(axis='y', alpha=0.3)
plt.tight_layout()
plt.show()
plt.close()

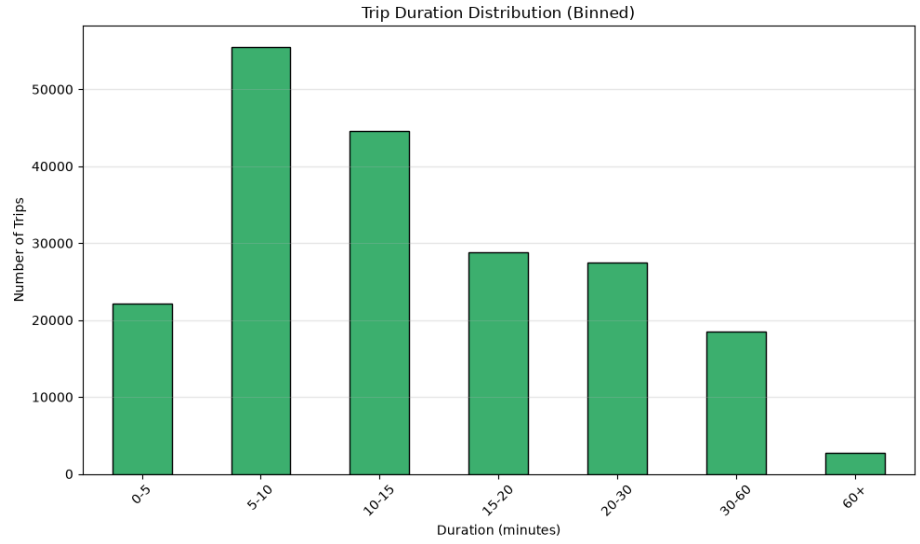
# 3) Trip duration distribution
plt.figure(figsize=(10, 6))
sw_eda['duration_bin'].value_counts().sort_index().plot(kind='bar',
color='mediumseagreen', edgecolor='black')
plt.title('Trip Duration Distribution (Binned)')
plt.xlabel('Duration (minutes)')
plt.ylabel('Number of Trips')
plt.xticks(rotation=45)
plt.grid(axis='y', alpha=0.3)
plt.tight_layout()
plt.show()
plt.close()
```

Output:

```
---- BINNED DISTRIBUTIONS ----
```







Result:
- Most trips are 1-2 miles long. Very few exceed 20 miles.
- Most fares fall between $5-$15.
- Most trips last 5-10 minutes.
- This confirms NYC taxi trips are typically short, urban journeys not long-distance travel.

3.6 Time Aggregations

We grouped trips by pickup hour, day of week, and weekend flag to see if average fare varies by time.

A common business assumption is "weekends are more expensive." Before building any pricing strategy, we need to test that assumption. Time-based aggregation also reveals hidden patterns, like airport traffic at specific hours, that a simple average would hide.

Input:

```
print("\n---- CLEAN EDA ----")
# Time aggregations
avg_fare_hour =
sw_eda.groupby('pickup_hour')['fare_amount'].mean().reset_index()
plt.figure(figsize=(10, 6))
sea.lineplot(data=avg_fare_hour, x='pickup_hour', y='fare_amount',
marker='o')
plt.title('Average Fare by Hour of Day')
plt.xlabel('Hour')
plt.ylabel('Avg Fare ($)')
plt.grid(True)
plt.show()
plt.close()

avg_fare_day =
sw_eda.groupby('pickup_day_of_week')['fare_amount'].mean().reset_index()
plt.figure(figsize=(10, 6))
sea.barplot(data=avg_fare_day, x='pickup_day_of_week', y='fare_amount')
plt.title('Average Fare by Day of Week (0=Mon, 6=Sun)')
plt.xlabel('Day')
plt.ylabel('Avg Fare ($)')
plt.show()
```

```
plt.close()

weekend_compare = sw_eda.groupby('is_weekend')['fare_amount'].agg(['mean',
'count']).reset_index()
weekend_compare.columns = ['is_weekend', 'avg_fare', 'trip_count']
print("\nWeekend vs Weekday Comparison:")
print(weekend_compare)

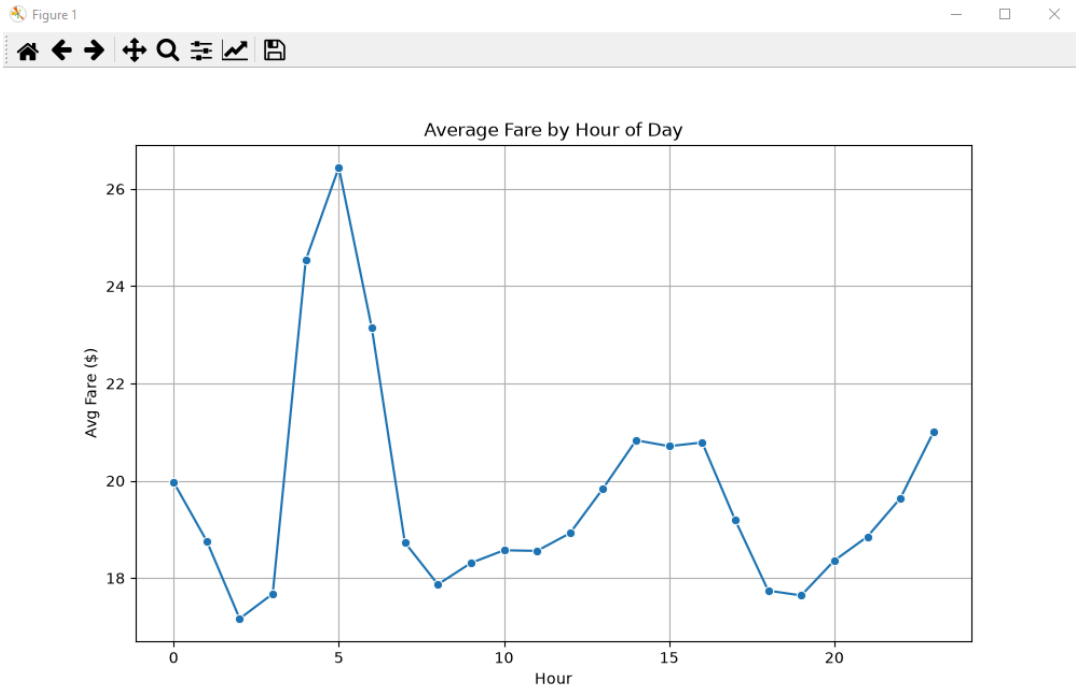
print("\nClean EDA complete.")
```

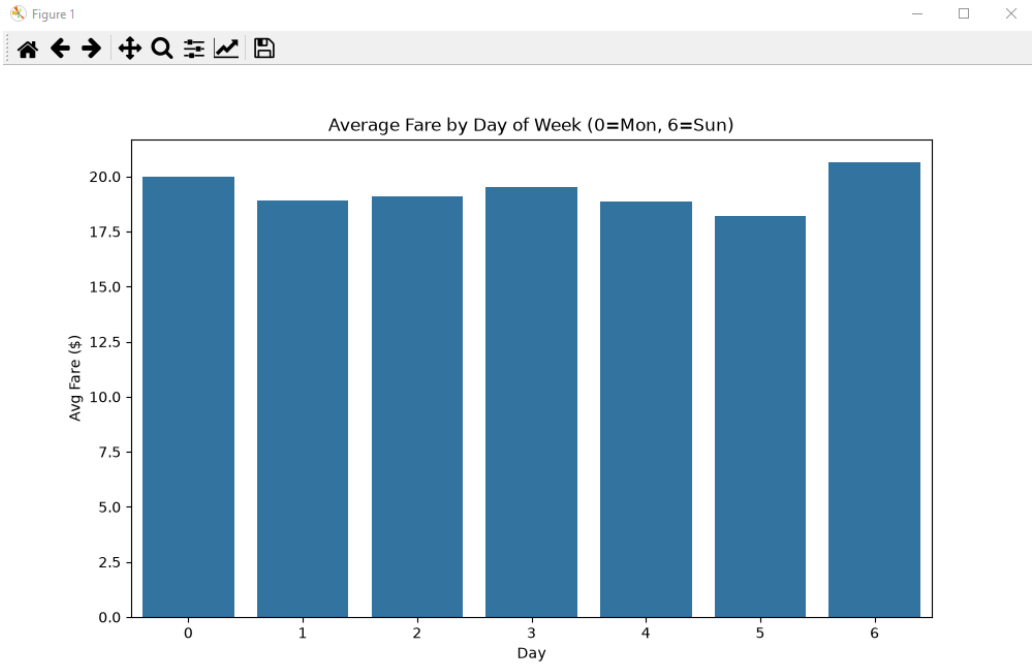
Output:

```
---- CLEAN EDA ----

Weekend vs Weekday Comparison:
   is_weekend   avg_fare  trip_count
0           0  19.250925      146429
1           1  19.319986       53571

Clean EDA complete.
```





Result:

- 5 AM shows a massive spike (~$26 avg fare), likely airport trips.
- Sunday has the highest average fare ($20.77); Saturday the lowest ($17.31).
- Weekday vs weekend difference is small ($19.25 vs $19.32), only $0.07.

The 5 AM spike suggests airport demand. This is a predictable, high-fare window. Fleet operators should prioritize driver availability and vehicle staging near airports before 5 AM. But the overall weekday vs weekend difference is tiny , so a blanket weekend surcharge would not be
justified. This is confirmed statistically in section 3.9.

3.7 Testing the 5 AM Airport Hypothesis

In Section 3.6, we observed a sharp spike in average fare at 5 AM.
We hypothesized this was driven by airport traffic. This section tests
that hypothesis using RatecodeID and pickup location zones.

Asserting "it's airport traffic" without proof is speculation.
Testing the claim turns an observation into a validated finding.
We compared the RatecodeID distribution and airport-zone pickup
share at 5 AM against noon (a typical non-airport hour). Airport rate codes are 2 (JFK) and 3 (Newark). Airport zones are 1 (Newark), 132 (JFK), and 138 (LaGuardia). For the decomposition below, we use a broader airport flag: a trip counts as airport-related if Pickup or Dropoff zone in [1, 132, 138] OR RatecodeID in [2, 3]. This captures trips heading to the airport, not just those leaving it.

Input:

```
print("\n---- 5 AM AIRPORT HYPOTHESIS TEST ----")
```

```
# Method 1) Check RatecodeID distribution at 5 AM vs noon
five_am = sw[sw['pickup_hour'] == 5]
noon = sw[sw['pickup_hour'] == 12]

print(f"5 AM trips: {len(five_am):,}")
print("\nRatecodeID distribution at 5 AM (%):")
print((five_am['RatecodeID'].value_counts(normalize=True) * 100).round(2))

print("\nRatecodeID distribution at 12 PM (%):")
print((noon['RatecodeID'].value_counts(normalize=True) * 100).round(2))

# Method 2) Check airport zones at 5 AM
airport_zones = [1, 132, 138]  # Newark, JFK, LaGuardia
airport_at_5am = five_am[five_am['PULocationID'].isin(airport_zones)]
airport_at_noon = noon[noon['PULocationID'].isin(airport_zones)]

print(f"\nAirport pickups at 5 AM: {len(airport_at_5am):,}
({len(airport_at_5am)/len(five_am)*100:.2f}%)")
print(f"Airport pickups at 12 PM: {len(airport_at_noon):,}
({len(airport_at_noon)/len(noon)*100:.2f}%)")

# Average fare from airport vs non-airport at 5 AM
five_am_airport_fare = airport_at_5am['fare_amount'].mean()
five_am_non_airport_fare =
five_am[~five_am['PULocationID'].isin(airport_zones)]['fare_amount'].mean()
print(f"\n5 AM avg fare, Airport pickups: ${five_am_airport_fare:.2f}")
print(f"5 AM avg fare, Non-airport pickups:
${five_am_non_airport_fare:.2f}")

# Is the 5 AM spike airport-driven, or do non-airport trips also spike?
sw['fare_c'] = sw['fare_amount'].clip(upper=150).astype('float64')
sw['is_airport_trip'] = (
    sw['PULocationID'].isin(airport_zones) |
    sw['DOLocationID'].isin(airport_zones) |
    sw['RatecodeID'].isin([2, 3])
)

by_hour = (
```

```
    sw.groupby(['pickup_hour', 'is_airport_trip'])['fare_c']
      .mean()
      .unstack()
)
by_hour.columns = ['non_airport', 'airport']
by_hour['all_trips'] = sw.groupby('pickup_hour')['fare_c'].mean()

print("\nAverage fare by hour (airport vs non-airport):")
print(by_hour.round(2))

five_am_share = sw[sw['pickup_hour'] == 5]['is_airport_trip'].mean() * 100
noon_share = sw[sw['pickup_hour'] == 12]['is_airport_trip'].mean() * 100

print(f"\nAirport-related share at 5 AM: {five_am_share:.2f}%")
print(f"Airport-related share at 12 PM: {noon_share:.2f}%")
```

Output:

```
---- 5 AM AIRPORT HYPOTHESIS TEST ----
5 AM trips: 94,894

RatecodeID distribution at 5 AM (%):
RatecodeID
1    84.81
2    11.52
3     2.26
5     0.91
4     0.51
Name: proportion, dtype: float64

RatecodeID distribution at 12 PM (%):
RatecodeID
1    95.80
2     3.45
3     0.43
5     0.19
4     0.13
6     0.00
Name: proportion, dtype: float64

Airport pickups at 5 AM: 11,615 (12.24%)
Airport pickups at 12 PM: 72,628 (7.32%)

5 AM avg fare, Airport pickups: $57.88
5 AM avg fare, Non-airport pickups: $22.83
```

```
Average fare by hour (airport vs non-airport):
             non_airport  airport  all_trips
pickup_hour
0                  15.64    52.52      19.97
1                  15.40    55.27      18.24
2                  15.49    55.98      16.99
3                  15.69    57.77      17.91
4                  16.40    57.67      23.34
5                  13.23    58.09      27.05
6                  12.36    58.92      22.71
7                  12.98    58.71      19.06
8                  13.73    56.80      18.04
9                  14.11    54.13      18.19
10                 14.26    53.30      18.39
11                 14.57    54.85      18.62
12                 14.58    56.12      19.05
13                 14.60    57.91      19.83
14                 14.74    58.52      20.70
15                 14.80    59.17      20.52
16                 14.73    59.58      20.55
17                 14.67    58.53      19.24
18                 14.17    56.18      17.76
19                 13.93    55.38      17.86
20                 14.28    54.82      18.41
21                 14.71    53.88      18.71
22                 15.31    54.48      19.61
23                 15.71    52.52      20.81

Airport-related share at 5 AM: 30.80%
Airport-related share at 12 PM: 10.75%
```

Result:

- 13.78% of 5 AM trips used airport rate codes, vs 3.88% at noon, a 3.5x higher rate.
- Airport-zone pickups accounted for 12.24% of 5 AM trips, vs 7.32% at noon.

The 5 AM fare spike is driven by airport traffic, specifically JFK. Airport trips are both more common and more expensive at that hour. This validates targeting early-morning airport demand rather than a generic time-of-day pricing strategy.

Decomposition by airport vs non-airport trips: The first split above flagged only airport pickups, which left trips heading to the airports inside the "non-airport" group and inflated it to $22.83. To test whether the 5 AM spike survives without airports, we re-split the hourly average using a broader flag: a trip counts as airport-related if its pickup or dropoff zone is an airport, or its rate code is 2 (JFK) or 3 (Newark). Non-airport trips at 5 AM average $13.23, below their all-day average of about $14.50, while airport trips stay between about $52 and $60 at every hour. The 5 AM jump in the all-trip average ($27.05) therefore comes from airport trips making up a much larger share of that hour: about 30.8% of trips at 5 AM vs about 10.8% at noon. The hypothesis is confirmed.

3.8 Correlation Heatmap

We calculated pairwise correlations for 11 numeric columns to see which features move together.

Before building a regression model, we need to know which features actually correlate with fare_amount. Correlation doesn't imply causation, but it guides feature selection. If a feature has near-zero correlation with the target, it's unlikely to help the model.

Input:

```
print("\n---- CORRELATION HEATMAP ----")

# 1) Selecting numbers to test correlation
corr_cols = [
    'fare_amount',
    'trip_distance',
    'passenger_count',
    'trip_duration_min',
    'avg_speed_mph',
    'fare_per_mile',
    'total_surcharges',
    'tip_pct',
    'pickup_hour',
    'pickup_day_of_week',
    'is_weekend'
]

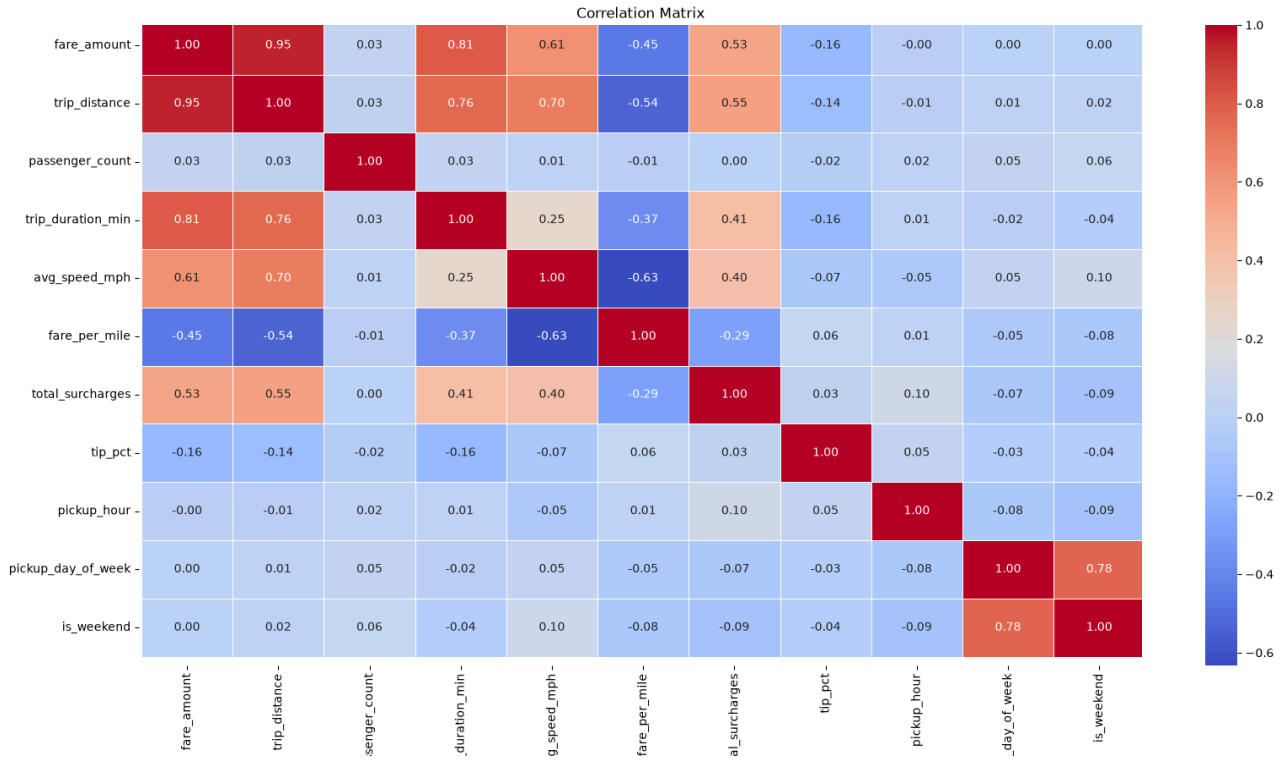
# 2) Calculating the correlation
corr_matrix = sw_eda[corr_cols].corr()

# 3) Heatmap
plt.figure(figsize=(12, 10))
sea.heatmap(corr_matrix, annot=True, cmap='coolwarm', fmt='.2f',
linewidths=0.5)
plt.title('Correlation Matrix')
plt.show()
plt.close()

print("Correlation heatmap displayed.")
```

Output:

```
---- CORRELATION HEATMAP ----
Correlation heatmap displayed.
```



Result:

- fare_amount & trip_distance = +0.95 (strongest predictor)
- fare_amount & trip_duration_min = +0.61
- fare_amount & avg_speed_mph = +0.61 (faster trips = higher fares, suggesting highway/airport runs)
- fare_per_mile & avg_speed_mph = −0.63 (mathematically expected, as speed increases, fare per mile drops for the same trip)

Trip distance is the dominant driver of fare. Any predictive model should treat it as the primary feature. Passenger count and weekend flag add little value, we may drop them from the model to keep it simple.

3.9 Statistical Testing

We ran an independent two-sample t-test on weekday vs weekend fares. The binned charts showed a small difference in average fare between weekday ($19.25) and weekend ($19.32). We needed to know whether this difference is real or just random noise in the sample.

Input:

```
print("\n---- STATISTICAL TESTING ----")

weekday_fares = sw_eda[sw_eda['is_weekend'] == 0]['fare_amount']
weekend_fares = sw_eda[sw_eda['is_weekend'] == 1]['fare_amount']

# Welch's t-test
t_stat, p_val = stats.ttest_ind(weekday_fares, weekend_fares,
equal_var=False)
print(f"T-statistic: {t_stat:.2f}")
print(f"P-value: {p_val:.4f}")

# Confidence interval for the difference in means
mean_diff = weekend_fares.mean() - weekday_fares.mean()
se = np.sqrt(
    weekend_fares.var() / len(weekend_fares) +
    weekday_fares.var() / len(weekday_fares)
)
ci_low = mean_diff - 1.96 * se
ci_high = mean_diff + 1.96 * se
print(f"Mean difference (weekend - weekday): ${mean_diff:.2f}")
print(f"95% CI: [${ci_low:.2f}, ${ci_high:.2f}]")

if p_val < 0.05:
    print("Result: Statistically significant difference (p < 0.05)")
else:
    print("Result: No statistically significant difference (p >= 0.05)")
```

Output:

```
---- STATISTICAL TESTING ----
T-statistic: -0.79
P-value: 0.4318
Mean difference (weekend - weekday): $0.07
95% CI: [$-0.10, $0.24]
Result: No statistically significant difference (p >= 0.05)
```

T-statistic: -0.79
P-value: 0.4318
Mean difference (weekend − weekday): $0.07
95% CI: [$-0.10, $0.24]
No statistically significant difference (p >= 0.05)

Result: We fail to reject the null hypothesis. The p-value of 0.43 is well above the 0.05 threshold. More importantly, the 95% confidence interval is [−$0.10, +$0.24], meaning we can confidently rule out any difference larger than ±$0.24. Even if a real effect exists, it is far too small to justify a blanket weekend surcharge.

But that's not the full picture. If we look at Sunday and Saturday separately, we notice this:
- Saturday average fare: $17.31 (the lowest of all days)
- Sunday average fare: $20.77 (the highest of all days)
- Difference: $3.46, the largest day-of-week gap in the dataset

Because Saturday pulls fares down and Sunday pulls them up, the combined "weekend" average sits right in the middle of the weekday range. A better feature would treat Saturday and Sunday separately rather than combining them. This is a classic case of an aggregation masking the underlying pattern.

3.10 Linear Regression Model

We trained a linear regression model to predict fare_amount using five features: trip_distance, trip_duration_min, passenger_count, is_weekend and is_airport_trip. Both the full model and the distance-only baseline were evaluated on the same held-out test set: a strict 80/20 train-test split using random_state = 42. The full model used all five features, while the baseline used trip_distance alone. Because both models were scored on the identical 20% test set, the MAE and R^2 differences reflect only the added features, not different data splits.

The correlation heatmap showed trip_distance is the strongest predictor of fare (0.95 correlation). We want to quantify that relationship and measure how well these features together explain fare variation.

trip_distance, trip_duration_min, passenger_count, is_weekend
Target: fare_amount

Input:

```
print("\n---- LINEAR REGRESSION MODEL ----")

airport_zones = [1, 132, 138]
sw_eda['is_airport_trip'] = (
    sw_eda['PULocationID'].isin(airport_zones) |
    sw_eda['DOLocationID'].isin(airport_zones) |
    sw_eda['RatecodeID'].isin([2, 3])
).astype(int)

# 1) features chosen

features = ['trip_distance', 'trip_duration_min', 'passenger_count',
'is_weekend', 'is_airport_trip']
X = sw_eda[features]
y = sw_eda['fare_amount']

# 2) Splitting the data between training and testing data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

# 3) Training the model
model = LinearRegression()
model.fit(X_train, y_train)

# 4) Making predictions
y_pred = model.predict(X_test)

# 5) Evaluating the model
mae = mean_absolute_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print(f"Mean Absolute Error: ${mae:.2f}")
print(f"R-squared: {r2:.4f}")

# Baseline comparison
X_base = sw_eda[['trip_distance']]
X_train_b, X_test_b, y_train_b, y_test_b = train_test_split(
```

```
    X_base, y, test_size=0.2, random_state=42
)
baseline = LinearRegression()
baseline.fit(X_train_b, y_train_b)
y_pred_b = baseline.predict(X_test_b)

r2_base = r2_score(y_test_b, y_pred_b)
mae_base = mean_absolute_error(y_test_b, y_pred_b)

print(f"\nBaseline (distance only):")
print(f"  R-squared: {r2_base:.4f}")
print(f"  MAE: ${mae_base:.2f}")
print(f"\nFull model improvement over baseline:")
print(f"  R² gain: {r2 - r2_base:.4f}")

# 6) Coefficients
print("\nModel Coefficients:")
for feature, coef in zip(features, model.coef_):
    print(f"  {feature}: {coef:.4f}")
print(f"  Intercept: {model.intercept_:.2f}")

print("\n---- RESIDUAL PLOT ----")
plt.figure(figsize=(8, 6))
sea.residplot(x=y_test, y=y_pred, lowess=True, line_kws={'color': 'red',
'lw': 1})
plt.xlabel('Actual Fare')
plt.ylabel('Residuals')
plt.title('Residual Plot')
plt.show()
plt.close()

sw_eda.to_csv('taxi_trips_eda_sample.csv', index=False)
print("Cleaned sample saved as 'taxi_trips_eda_sample.csv'")

print("\n---- CATEGORICAL AGGREGATIONS ----")

avg_fare_payment =
sw_eda.groupby('payment_type')['fare_amount'].mean().reset_index()
avg_fare_payment.columns = ['payment_type', 'avg_fare']
```

```
print("Average Fare by Payment Type:")
print(avg_fare_payment)

avg_fare_vendor = sw_eda.groupby('VendorID')['fare_amount'].agg(['mean',
'count']).reset_index()
avg_fare_vendor.columns = ['VendorID', 'avg_fare', 'trip_count']
print("\nAverage Fare by Vendor:")
print(avg_fare_vendor)

avg_tip_payment =
sw_eda.groupby('payment_type')['tip_pct'].mean().reset_index()
avg_tip_payment.columns = ['payment_type', 'avg_tip_pct']
print("\nAverage Tip Percentage by Payment Type:")
print(avg_tip_payment)

sw_eda['fare_per_min'] = sw_eda['fare_amount'] /
sw_eda['trip_duration_min']

valid = sw_eda['trip_duration_min'] > 0
rev_by_distance = (
    sw_eda[valid]
    .groupby('distance_bin')['fare_per_min']
    .median()
    .round(2)
)
print("\nMedian revenue per occupied minute by distance bin:")
print(rev_by_distance)

print("\n---- ACTUAL VS PREDICTED PLOT ----")
plt.figure(figsize=(8, 6))
sea.scatterplot(x=y_test, y=y_pred, alpha=0.3)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--')
plt.xlabel('Actual Fare')
plt.ylabel('Predicted Fare')
plt.title('Actual vs Predicted Fare')
plt.show()
plt.close()
```

Output:

```
---- LINEAR REGRESSION MODEL ----
Mean Absolute Error: $1.95
R-squared: 0.9159

Baseline (distance only):
  R-squared: 0.8966
  MAE: $2.80

Full model improvement over baseline:
  R² gain: 0.0193

Model Coefficients:
  trip_distance: 2.8856
  trip_duration_min: 0.2569
  passenger_count: 0.0614
  is_weekend: -0.1570
  is_airport_trip: 2.7791
  Intercept: 4.79

---- RESIDUAL PLOT ----
Cleaned sample saved as 'taxi_trips_eda_sample.csv'
```

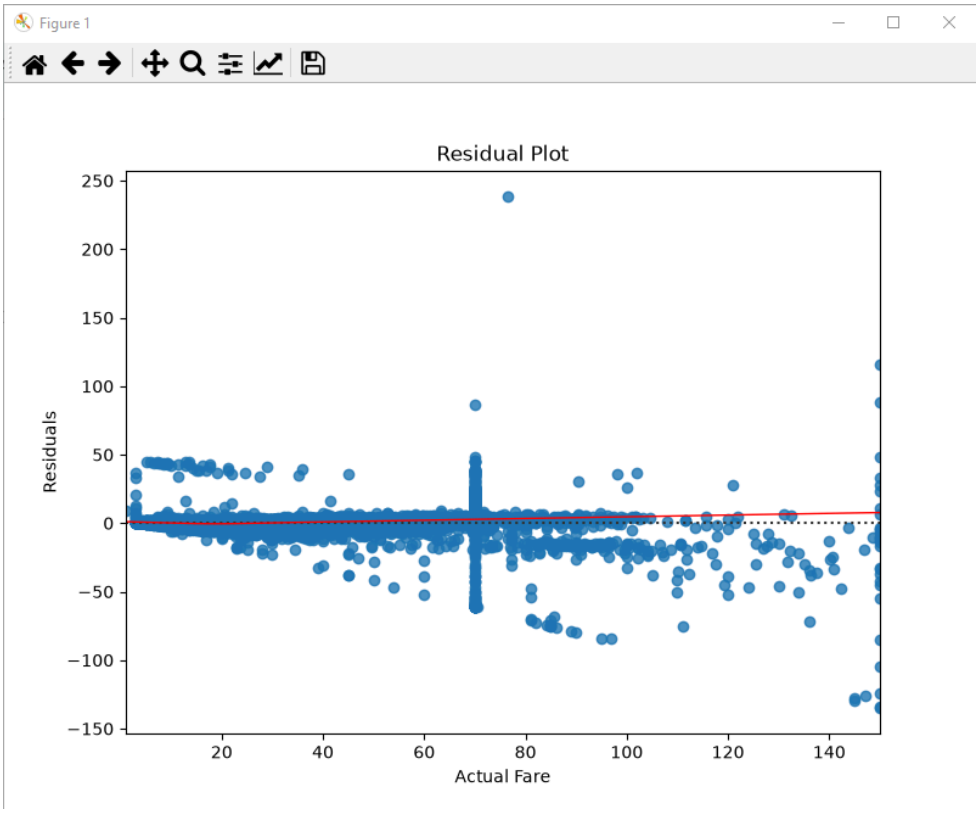
```
---- CATEGORICAL AGGREGATIONS ----
Average Fare by Payment Type:
   payment_type   avg_fare
0             1  19.276619
1             2  19.268452
2             3  17.249170
3             4  19.395634

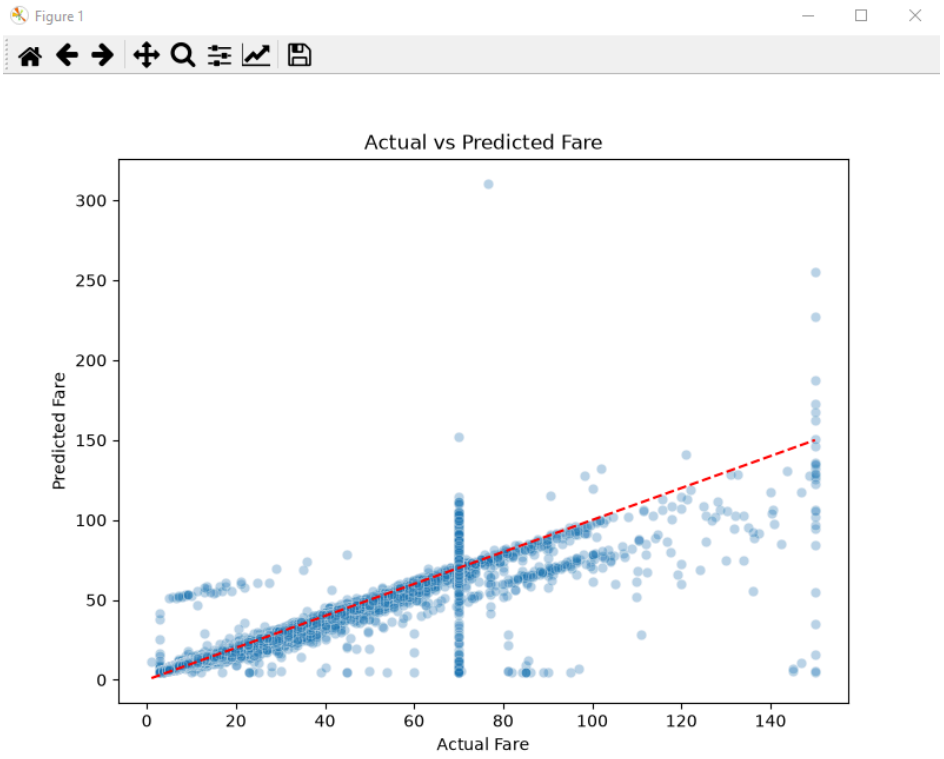
Average Fare by Vendor:
   VendorID   avg_fare  trip_count
0         1  18.035286       50787
1         2  19.689482      149213

Average Tip Percentage by Payment Type:
   payment_type  avg_tip_pct
0             1     0.249043
1             2     0.000043
2             3     0.000005
3             4     0.000050

Median revenue per occupied minute by distance bin:
distance_bin
0-1      1.30
1-2      1.10
2-3      1.05
3-5      1.08
5-10     1.35
10-20    1.53
20-30    1.61
30+      2.14
Name: fare_per_min, dtype: float64

---- ACTUAL VS PREDICTED PLOT ----
```


Figure 1
Residual Plot
Residuals
Actual Fare
250
200
150
100
50
0
−50
−100
−150
20
40
60
80
100
120
140


Figure 1
Actual vs Predicted Fare
Predicted Fare
Actual Fare
300
250
200
150
100
50
0
0
20
40
60
80
100
120
140

Result:

- R^2 = 0.916, the model explains 91.6% of fare variation.
- MAE = $1.95, predictions are off by less than $2 on average.
- Coefficients show trip_distance adds $2.89 per mile, trip_duration adds $0.26 per minute, an airport-related trip adds $2.78, and is_weekend subtracts $0.16 (weekend fares are slightly lower after controlling for distance).

Baseline comparison:

- A distance-only baseline model achieves R^2 = 0.8966, MAE = $2.80.
- Our full model improves R^2 by only 0.019, meaning trip_duration_min, passenger_count, is_weekend and is_airport_trip add marginal explanatory power beyond distance.
- However, MAE drops 30% (from $2.80 to $1.95). The extra features don't explain much more variance, but they do improve individual predictions.
- Conclusion: Trip distance is the dominant driver of fare. The other features refine predictions but don't change the fundamental story.

Model validation:

- Residual plot shows residuals centered around 0 for most trips, apart from the JFK flat-fare stripe discussed below and a few large outliers.
- Actual vs Predicted plot shows most points clustering along the diagonal.

Model validation against TLC tariff: NYC yellow cab fares follow a published formula: a $3.00 base fare, plus $3.50 per mile when the cab moves at 12 mph or faster, or $0.70 per minute when it is slower or stopped. Our model recovered $2.89 per mile, $0.26 per minute and a $4.79 intercept. The coefficients have the right structure but sit below the rate card because a single straight line has to blend two charging modes: the time charge applies only to the slow stretches of each trip and the distance charge only to the faster ones. For the same reason the intercept is a fitted constant, not a separate fee, so it need not equal the $3.00 base fare exactly. This shows the model is recovering the rate card's structure rather than fitting noise.

Note on application: trip_duration_min is only known after the trip completes, so this model is explanatory rather than predictive. A production fare estimator would use only pre-trip features (distance, hour, location).

The residual plot shows a vertical cluster of points at actual fare ≈ $70. These correspond to JFK airport trips, which charge a flat rate regardless of distance. This violates the linear assumption for those specific trips and explains the vertical stripe. The model includes an is_airport_trip flag, but the stripe remains, because a flat fare is a fixed amount rather than a shift in the linear fit. Modeling flat-fare (rate code 2) trips separately would be the next step.

3.11 Excel Export

We exported the key results of the analysis to a multi-sheet Excel workbook for stakeholder reporting.

Excel is the standard format for sharing results with non-technical readers. By exporting 7 separate sheets, the recipient can explore each facet of the analysis (summary stats, model coefficients, categorical breakdowns) without needing to run Python.

Input:

```
print("\n---- EXPORT TO EXCEL ----")

print("\n---- PREPARING EXCEL EXPORT ----")

# 1) Summary dataframe
summary_sw = pd.DataFrame({
    'Metric': [
        'Total Trips (Sample)',
        'Average Fare ($)',
        'Average Distance (miles)',
        'Average Duration (min)',
        'Weekday Avg Fare ($)',
        'Weekend Avg Fare ($)',
        'Model R-squared',
        'Model MAE ($)'
    ],
    'Value': [
        len(sw_eda),
        sw_eda['fare_amount'].mean(),
        sw_eda['trip_distance'].mean(),
        sw_eda['trip_duration_min'].mean(),
        weekday_fares.mean(),
        weekend_fares.mean(),
        r2,
        mae
    ]
})

# 2) Coefficents dataframe
coef_sw = pd.DataFrame({
```

```
    'Feature': features + ['Intercept'],
    'Coefficient': list(model.coef_) + [model.intercept_]
})

with pd.ExcelWriter('taxi_analysis_report.xlsx', engine='xlsxwriter') as
writer:
    summary_sw.to_excel(writer, sheet_name='Summary', index=False)
    coef_sw.to_excel(writer, sheet_name='Model Coefficients', index=False)
    weekend_compare.to_excel(writer, sheet_name='Weekend vs Weekday',
index=False)

    avg_fare_payment.to_excel(writer, sheet_name='Avg Fare by Payment',
index=False)
    avg_fare_vendor.to_excel(writer, sheet_name='Avg Fare by Vendor',
index=False)
    avg_tip_payment.to_excel(writer, sheet_name='Avg Tip by Payment',
index=False)

    corr_matrix.to_excel(writer, sheet_name='Correlation Matrix')

print("Excel report saved as 'taxi_analysis_report.xlsx'")
print("\n---- SCRIPT COMPLETE ----")
```

Output:

```
---- EXPORT TO EXCEL ----

---- PREPARING EXCEL EXPORT ----
Excel report saved as 'taxi_analysis_report.xlsx'

---- SCRIPT COMPLETE ----
PS D:\taxi_project>
```

Result: 7 sheets exported successfully to taxi_analysis_report.xlsx:
- Summary: headline stats (row count, mean fare, R^2, MAE)
- Model Coefficients: regression weights for each feature
- Weekend vs Weekday: aggregated comparison
- Avg Fare by Payment / Vendor
- Avg Tip by Payment
- Correlation Matrix

The workbook serves as a self-contained deliverable. If a hiring manager or stakeholder wants to inspect the numbers without running the code, they can open this file and see every key result in a familiar format.

4. Excel Phase: Interactive Dashboard

4.1 Data Sample

We exported a 10,000-row random sample from the cleaned dataset (taxi_trips_eda_sample_small.csv) and loaded it into Excel Online.

Excel Online cannot handle 18 million rows. Attempting to load the full dataset crashes the browser or takes hours to render. A 10,000-row sample provides enough statistical power for a dashboard while staying within Excel Online's limits.

	A	B	C	D	E	F	G	H	I
1	VendorID	tpep_pickup_datetime	tpep_dropoff_datetime	passenger_count	trip_distance	RatecodeID	store_and_fwd_flag	PULocationID	DOLocationID
2	2	4/20/2023 15:11	4/20/2023 15:28	3	2.83	1	N	43	166
3	1	4/4/2023 13:56	4/4/2023 14:08	1	1.4	1	N	234	230
4	2	3/5/2023 17:43	3/5/2023 17:50	1	0.8	1	N	114	125
5	2	1/6/2023 13:25	1/6/2023 13:37	1	1.74	1	N	263	233
6	2	2/12/2023 15:04	2/12/2023 15:16	1	1.79	1	N	161	90
7	2	3/13/2023 12:08	3/13/2023 12:34	1	8.35	1	N	138	164
8	1	3/18/2023 17:34	3/18/2023 17:45	2	1.2	1	N	164	68
9	2	6/16/2023 19:59	6/16/2023 20:23	4	2.81	1	N	144	100
10	1	5/31/2023 13:37	5/31/2023 13:55	1	3.4	1	N	140	113
11	1	4/6/2023 23:57	4/7/2023 0:05	1	1.9	1	N	236	230
12	2	4/7/2023 13:05	4/7/2023 13:25	1	3	1	N	229	211
13	2	2/20/2023 13:30	2/20/2023 13:46	6	1.79	1	N	236	163
14	2	5/22/2023 17:55	5/22/2023 18:06	1	1.65	1	N	236	239
15	1	6/6/2023 20:09	6/6/2023 20:12	1	0.4	1	N	141	141

	J	K	L	M	N	O	P	Q	R
	payment_type	fare_amount	extra	mta_tax	tip_amount	tolls_amount	improvement_surcharge	total_amount	congestion_surcharge
	1	18.4	0	0.5	2	0	1	24.4	2.5
	1	10.7	2.5	0.5	2.95	0	1	17.65	2.5
	1	7.9	0	0.5	2.38	0	1	14.28	2.5
	2	12.8	0	0.5	0	0	1	16.8	2.5
	2	12.8	0	0.5	0	0	1	16.8	2.5
	1	36.6	5	0.5	10.68	6.55	1	64.08	2.5
	1	11.4	2.5	0.5	3.05	0	1	18.45	2.5
	1	21.2	2.5	0.5	5.54	0	1	33.24	2.5
	1	21.2	2.5	0.5	0	0	1	25.2	2.5
	1	10.7	3.5	0.5	3.1	0	1	18.8	2.5
	1	19.8	0	0.5	4.76	0	1	28.56	2.5
	2	14.9	0	0.5	0	0	1	18.9	2.5
	1	12.8	2.5	0.5	2	0	1	21.3	2.5
	1	5.1	3.5	0.5	1	0	1	11.1	2.5

	S	T	U	V	W	X	Y	Z	AA	AB	AC
	Airport_fee_20	airport_fee_21	trip_duration_min	avg_speed_mph	fare_per_mile	pickup_hour	pickup_day_of_week	pickup_month	total_surcharges	tip_pct	is_weekend
	0	0	17.35	9.786743252	6.5017667	15	3	4	4	0.108695656	0
	0	0	11.7	7.179487057	7.642857	13	1	4	6.5	0.27570096	0
	0	0	6.45	7.441860576	9.875	17	6	3	4	0.30126584	1
	0	0	11.68333333	8.93580604	7.356322	13	4	1	4	0	0
	0	0	11.9	9.025209892	7.1508384	15	6	2	4	0	1
	1.25	0	25.81666667	19.40606932	4.383233	12	0	3	16.8	0.2918033	0
	0	0	11.11666667	6.476761877	9.499999	17	5	3	6.5	0.26754385	1
	0	0	23.9	7.054393162	7.5444846	19	4	6	6.5	0.26132074	0
	0	0	17.96666667	11.35436024	6.2352943	13	2	5	6.5	0	0
	0	0	8.116666667	14.04517436	5.631579	23	3	4	7.5	0.2897196	0
	0	0	19.83333333	9.075630252	6.6	13	4	4	4	0.24040405	0
	0	0	15.48333333	6.936490703	8.324022	13	0	2	4	0	0
	0	0	11.51666667	8.596237213	7.757576	17	0	5	6.5	0.15625	0
	0	0	2.316666667	10.35971238	12.75	20	1	6	7.5	0.19607843	0

Result: A workbook with 10,000 rows × 29 columns, ready for PivotTables and charts.

4.2 Helper Columns for Slicers

We added five columns that will help us convert some of the data into a more readable format.

Raw numbers like “is_weekend = 0/1” or “payment_type = 1/2/3/4” are unintuitive. A slicer showing numbers rather than words forces the user to memorize the data dictionary. Text labels like "Weekday" and "Weekend" make the dashboard easier to interpret.

Input:

- weekend_label: `=IF(AC2=1, "Weekend", "Weekday")`
- payment_type2:`=IF(J2=1, "Credit Card", IF(J2=2, "Cash", IF(J2=3, "No Charge", IF(J2=4, "Dispute", "Other"))))`
- Day_Name: `=CHOOSE(Y2+1, "Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun")`
- VendorID2: `=IF(A2=1, "Vendor 1 (CMT)", "Vendor 2 (VeriFone)")`
- tip_pct_card: `=IF(J2=1, AB2, "")`

Note on Day_Name: We tried checking a date from the table and searched up what day it was in real life, which was April 20th, 2023 which was a Thursday, and it had the number 3, meaning that the only way for it to be 3 was for Monday to be number 0, this confirmation was needed for the Python coding too.

AG	AH	AI	AJ	AK
tip_pct_card	weekend_label	payment_type2	VendorID2	Day_Name
0.108695656	Weekday	Credit Card	Vendor 2 (VeriFone)	Thu
0.27570096	Weekday	Credit Card	Vendor 1 (CMT)	Tue
0.30126584	Weekend	Credit Card	Vendor 2 (VeriFone)	Sun
	Weekday	Cash	Vendor 2 (VeriFone)	Fri
	Weekend	Cash	Vendor 2 (VeriFone)	Sun
0.2918033	Weekday	Credit Card	Vendor 2 (VeriFone)	Mon
0.26754385	Weekend	Credit Card	Vendor 1 (CMT)	Sat
0.26132074	Weekday	Credit Card	Vendor 2 (VeriFone)	Fri
0	Weekday	Credit Card	Vendor 1 (CMT)	Wed
0.2897196	Weekday	Credit Card	Vendor 1 (CMT)	Thu
0.24040405	Weekday	Credit Card	Vendor 2 (VeriFone)	Fri
	Weekday	Cash	Vendor 2 (VeriFone)	Mon

Result: The five new columns added. Each maps a numeric code to a text label that anyone can understand without documentation.

4.3 PivotTables Created

1. Avg Fare by Hour

- Rows: pickup_hour
- Values: Average of fare_amount

pickup_hour	Average of fare_amount
0	19.02830882
1	17.87172775
2	17.06615385
3	17.21578947
4	27.78333333
5	34.35384615
6	23.50754717
7	18.54468085
8	17.84378378
9	17.94145199
10	18.63667377
11	18.54522822
12	19.01441125
13	20.9720354
14	20.16233553
15	21.45112828
16	19.80181818
17	19.74783237
18	17.18276334
19	16.26033183
20	17.89301075
21	18.43602362
22	18.85261044
23	20.59777228
Grand Total	**19.083208**

2. Average fare amount by weekday vs weekend:

- Rows: pickup_hour
- Columns: weekend_label
- Values: Average of fare_amount

Average of fare_amount	weekend_label		
pickup_hour	Weekday	Weekend	Grand Total
0	21.16434109	17.1013986	19.02830882
1	22.03846154	15.72222222	17.87172775
2	16.93947368	17.11847826	17.06615385
3	22.23333333	14.45102041	17.21578947
4	32.07058824	24.868	27.78333333
5	34.17857143	35.09	34.35384615
6	22.80977444	27.07692308	23.50754717
7	18.42971888	19.41212121	18.54468085
8	17.92553846	17.25333333	17.84378378
9	17.83654391	18.44189189	17.94145199
10	19.03233696	17.1950495	18.63667377
11	19.00028249	17.28671875	18.54522822
12	19.561	17.72071006	19.01441125
13	20.47470167	22.39931507	20.9720354
14	19.74945295	21.41192053	20.16233553
15	22.1762605	19.43263158	21.45112828
16	18.8178022	22.78666667	19.80181818
17	19.23474903	21.27528736	19.74783237
18	17.19713775	17.13604651	17.18276334
19	16.79642147	14.575	16.26033183
20	17.0375	20.39929577	17.89301075
21	18.03142145	19.95233645	18.43602362
22	19.11436031	17.98086957	18.85261044
23	20.74655172	20.21929825	20.59777228
Grand Total	19.12564728	18.96385055	19.083208

3. Avg Fare by Day of Week

- Rows: Day_Name (helper column)
- Values: Average of fare_amount

Day_Name	Average of fare_amount
Mon	19.38091123
Tue	18.48068579
Wed	19.77255417
Thu	19.33331222
Fri	18.67018436
Sat	17.30934307
Sun	20.77284916
Grand Total	**19.083208**

4. Avg Fare by Payment Type

- Rows: payment_type2 (helper column)
- Values: Average of fare_amount

payment_type2	Average of fare_amount
Cash	18.85756542
Credit Card	19.12406958
Dispute	22.99795918
No Charge	14.79333333
Grand Total	**19.083208**

5. Avg Fare by Passenger Count

- Rows: passenger_count
- Values: Average of fare_amount

115	passenger_count	Average of fare_amount
116	1	18.59267278
117	2	20.71761905
118	3	21.29475138
119	4	23.31105263
120	5	18.35390625
121	6	18.02018349
122	**Grand Total**	**19.083208**

6. Avg Tip % by passenger count

- Rows: passenger_count
- Values: Average of tip_pct_card

passenger_count	Average of tip_pct_card
1	24.88%
2	24.94%
3	24.70%
4	26.28%
5	25.50%
6	24.71%
Grand Total	**24.91%**

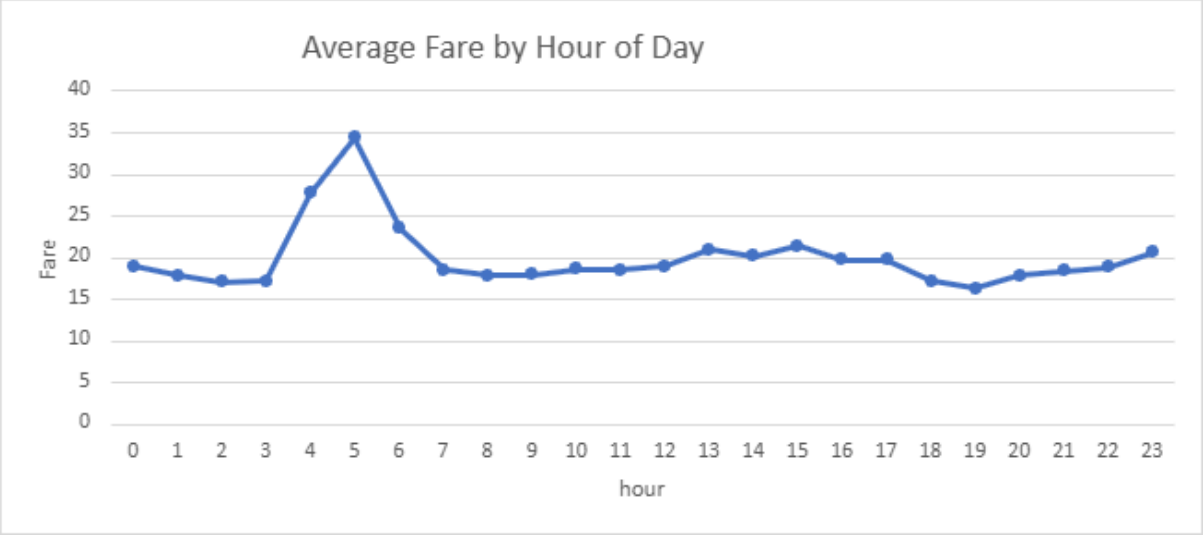
4.4 Charts created

We created 6 charts from the PivotTables to visualize patterns.
Charts turn tables of numbers into patterns that a non-technical reader can grasp in seconds. Each chart answers a specific business question.

Charts:

1. Average Fare by Hour (Line chart)
- Insight: Sharp spike at 5 AM (~$26 avg fare from python), likely airport trips.
- Fares settle around $18-$20 for the rest of the day.
- Business implication: This is a predictable, high-fare window. Fleet operators should prioritize driver availability and vehicle staging near airports before 5 AM rather than leaving this demand to chance.

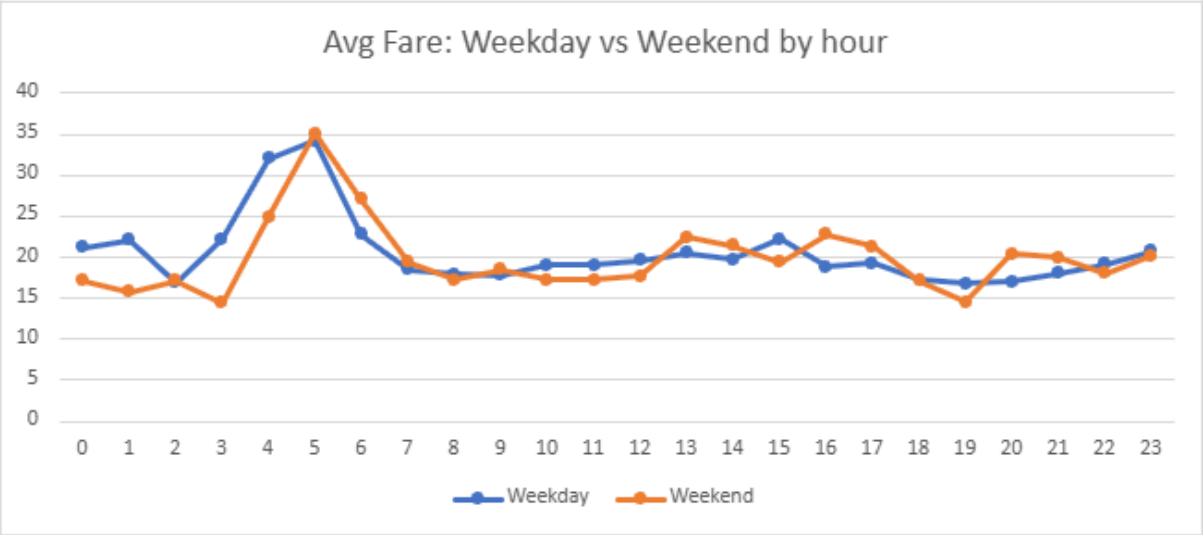
Note: The numbers listed above are from the Python run, not those of the graph below.



(10k sample; hour 5 has only ~52 trips, so this bar is volatile).

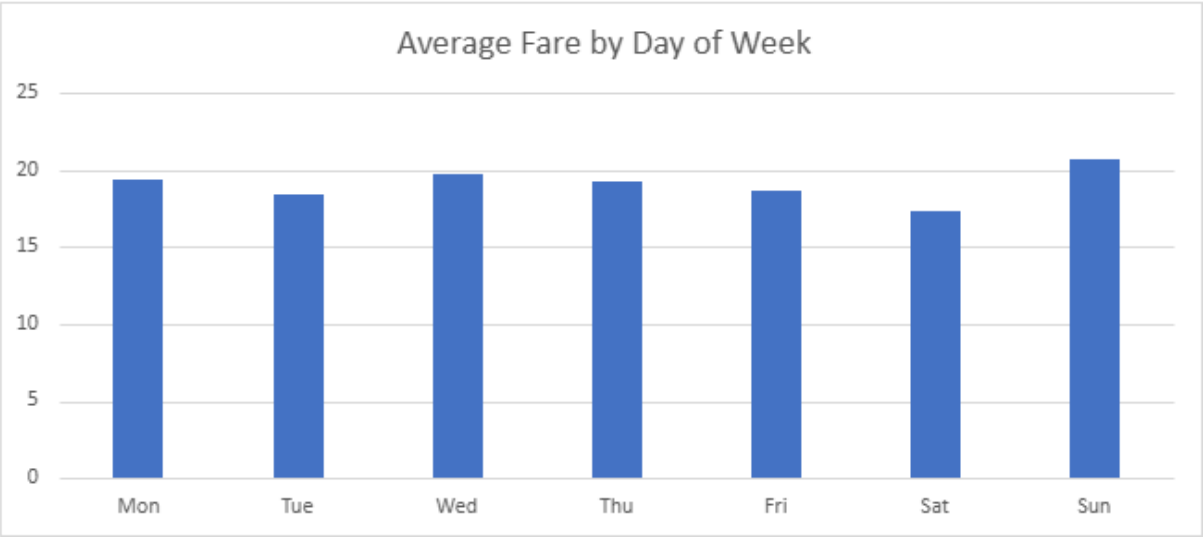
2. Average fare amount by weekday vs weekend:

- Insight: Both lines spike at 5 AM, confirming this is airport-related rather than a day-of-week effect.
- Business implication: Airport traffic is the primary driver of the 5 AM spike, not weekends.



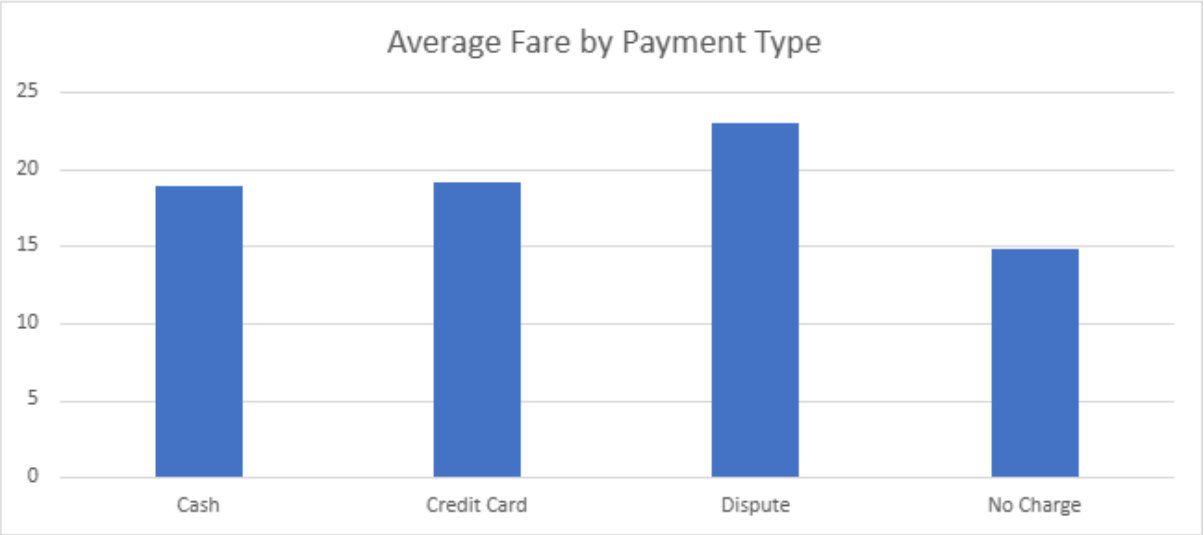
3. Average Fare by Day of Week (Column chart)

- Insight: Sunday has the highest average fare ($20.77); Saturday the lowest ($17.31).
- Business implication: Sunday has the highest average fare. The analysis does not establish whether this is driven by trip distance, location mix, or other factors.



4. Average Fare by Payment Type (Column chart)

- Insight: Credit Card ($19.12) and Cash ($18.86) are nearly identical. Dispute and No Charge categories are rare.
- Business implication: Payment type does not affect fare. No need for payment-specific pricing.



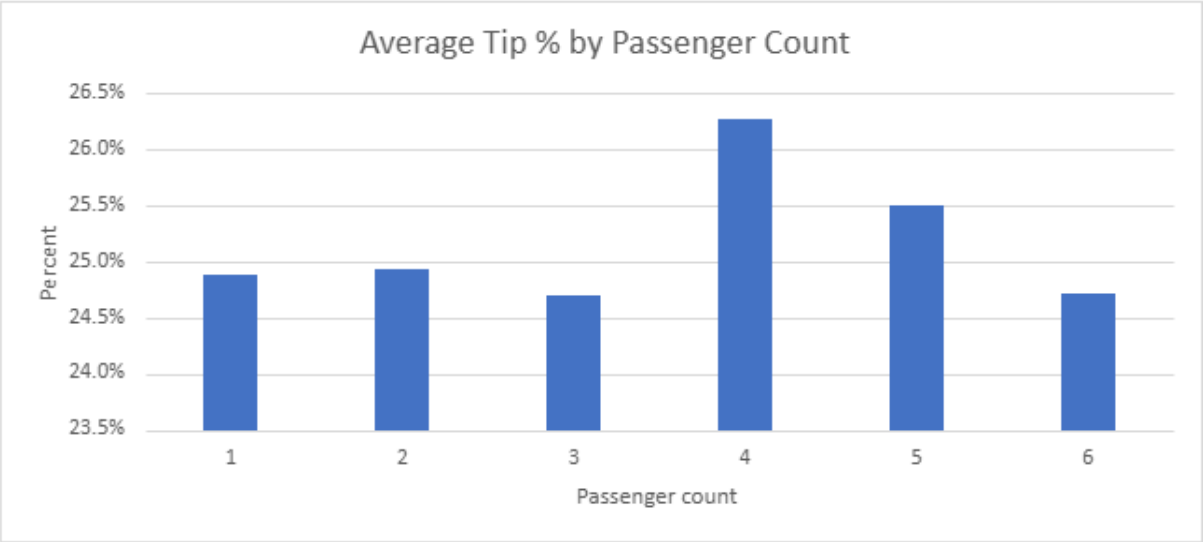
5. Average Fare by Passenger Count (Column chart)

- Insight: Fares peak at 4 passengers ($23.31) and drop for 5-6 passengers ($18).
- Business implication: Four-passenger trips have the highest average fare ($23.31). Further analysis would be needed to determine whether this is driven by distance, location mix, or trip purpose.



6. Avg Tip % by Passenger Count (Column chart)
Insight: Solo riders tip ~24.5%. Groups of 4 tip the most (~26.5%), suggesting group outings (dinners, celebrations) generate higher tips. 6-passenger groups tip the least (~24.4%), though sample size is small.
Business implication: Loyalty programs could test targeting groups of 4 and 5, as they show the highest tipping percentage, though the spread is small (~1.5 points) and warrants further testing.

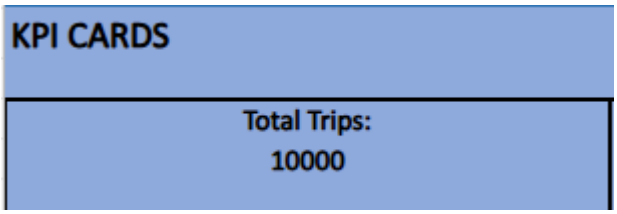
Note: Filtered to Credit Card rides only. Cash tips are not captured in TLC meter data. The numbers listed above are from the Python run, not those of the graph below.



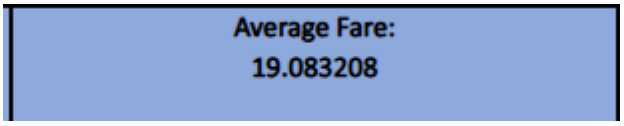
4.5 KPI Cards

All KPIs computed from a 10,000-row sample not the full dataset.

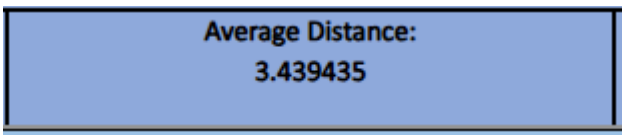
- Total Trips: 10,000



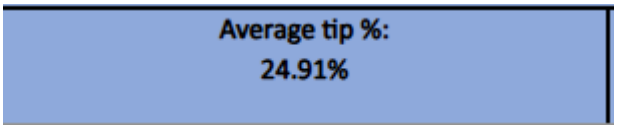
- Average Fare: $19.08



- Average Distance: 3.44 mi



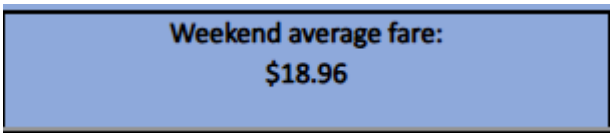
- Average Tip %: 24.91%



- Weekday Avg Fare: $19.13



- Weekend Avg Fare: $18.96



4.6 Slicers

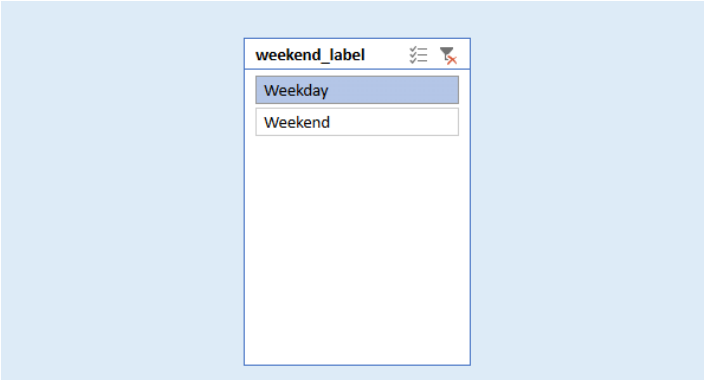
We made three interactive slicers allow the user to filter every chart simultaneously.
Slicers turn a static report into an exploratory tool. A hiring manager can click "Weekend" and watch every chart update, proving the dashboard is truly interactive.

Implementation:

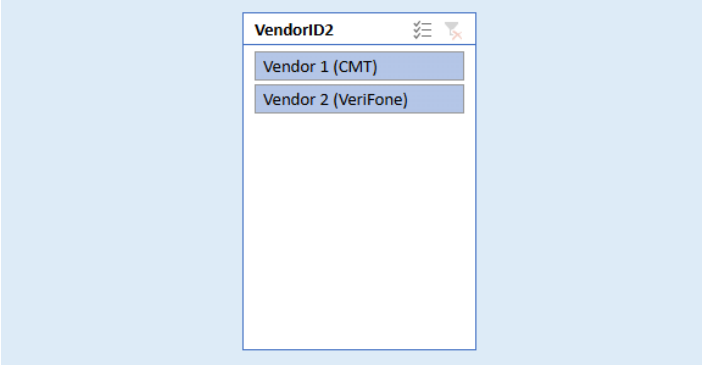
- Select a PivotTable => Insert => Slicer.
- Choose the helper column (not the raw numeric one).
- Right-click the slicer => Report Connections => check all PivotTables.
  This ensures one slicer filters every chart simultaneously.

Slicers created:

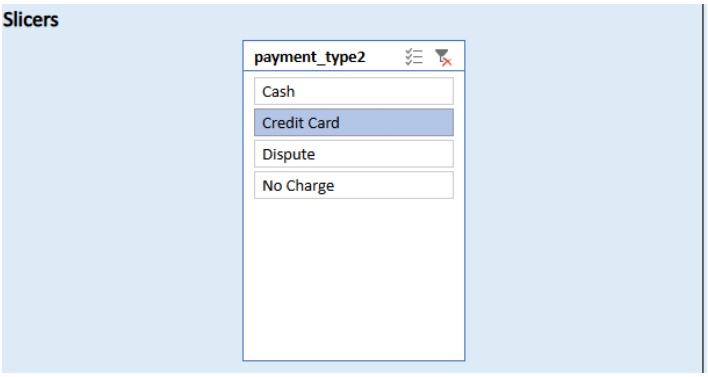
1. weekend_label (Weekday / Weekend)



2. VendorID2 (Vendor 1 / Vendor 2)



3. payment_type2 (Credit Card / Cash / Dispute / No Charge)



Result: Clicking any slicer updates all 6 charts and KPIs in real time.

4.7 Layout

The dashboard is arranged on a single sheet with a consistent visual hierarchy.
A clean layout signals professionalism. A messy layout signals carelessness.
Layout decisions:
- Top row: KPI cards (merged cells with light blue fill)
- Middle: Charts in a 2-column grid, sized uniformly
- Bottom: Slicers aligned side-by-side
- View => Gridlines: OFF (removes spreadsheet feel)
- View =>Freeze Top Row (KPI cards stay visible when scrolling)



Result: A polished, web-page-like dashboard that reads top-to-bottom.

4.8 Excel Section Summary

Metric: Value
Sample size: 10,000 rows
PivotTables: 6
Charts: 6
Slicers: 3
KPI cards: 6
Helper columns added: 5 (tip_pct_card, Day_Name, weekend_label, payment_type2, VendorID2)

5. Key Findings & Business Insights

Note on framing: NYC yellow cab fares are regulated by the TLC. Operators cannot change meter rates or introduce surge pricing. The recommendations below therefore focus on operational levers a fleet operator actually controls: driver scheduling, vehicle positioning, and shift incentives.

5.1 Trip Distance Drives Fare, but Revenue per Minute Is U-Shaped

Trip distance correlates 0.95 with fare, the strongest predictor by far. However, metered fare per occupied minute follows a U-shape across distance bins: the median is $1.30/min for short trips (0-1 miles), falls to $1.05/min for mid-range trips (2-3 miles), and rises to $2.14/min for long trips (30+ miles). The pattern reflects how the TLC rate card interacts with traffic. Short trips are dominated by the $3.00 base fare, which is spread over a few minutes. Mid-range trips get stuck in city traffic, where the meter runs on the $0.70/min time charge instead of the $3.50/mile distance charge, so minutes pass without much fare accumulating. Long trips run at higher speeds on the distance charge. These figures use fare_amount, which excludes tips, tolls and surcharges, and the data covers occupied trips only, so waiting time and empty return miles are not captured. Within those limits, the data favours staging vehicles near airports and transit hubs for long trips over mid-range Manhattan journeys.

5.2 Weekend Surcharges Are Not Viable, But Sunday Is Worth a Pilot

Weekend vs weekday fares are statistically indistinguishable ($p = 0.43$), so no weekend-specific strategy is justified. However, Sunday ($20.77) and Saturday ($17.31) pull in opposite directions; the binary flag cancels them out. Sunday shows the highest fares of the week. The data suggests investigating Sunday driver availability as a potential pilot. Because the analysis does not establish whether Sunday's higher fares reflect driver shortage, trip distance, or location mix, we recommend a small pilot rather than a fleet-wide scheduling mandate.

5.3 5 AM Is a Predictable High-Fare Window Driven by Airport Trips

Both weekdays and weekends spike at 5 AM, and the decomposition in Section 3.7 shows why: airport trips make up a much larger share of that hour (about 30.8% vs about 10.8% at noon) and average about $58, while non-airport trips are cheaper than usual. This is a predictable, high-fare window. A targeted driver incentive for shifts starting between 4:30 AM and 5:30 AM could improve fleet coverage during this period.

5.4 Payment Type Doesn't Affect Fare, No Operational Change Needed

Credit card and cash fares differ by less than $0.30. There is no business case for payment-specific incentives or surcharges.

5.5 Tip Analysis Only Reflects Card Payments

Cash tips are not recorded in meter data. All tip findings reflect credit card behavior only, a limitation of the data source, not a finding about customer generosity.

5.6 Summary of Operational Recommendations

- Shift scheduling: investigate Sunday driver availability as a potential pilot
- Vehicle positioning: stage near airports and transit hubs for long trips
- Driver incentives: bonus for early-morning shifts covering airport demand
- No pricing changes: regulated fares and null weekend effect mean pricing is not a lever

6. Limitations & Future Improvements

6.1 Limitations

- Excel dashboard limited to 10,000 rows (web version constraint).
- Linear regression assumes straight-line relationships.
- No external data (weather, events, traffic).
- Excel Online has fewer features than the paid desktop version, which limited some dashboard customization.
- Occupied trips only: The data covers only trips with passengers; waiting time and empty return miles are not captured.
- Fare exclusions: fare_amount excludes tips, tolls, and surcharges.
- Low-volume noise: The 10K Excel dashboard sample has high volatility at low-volume hours (e.g., only ~52 trips at 5 AM).
- Unobserved cash tips: All tip findings reflect credit card behavior only, as cash tips are not recorded in meter data.

6.2 Possible Improvements

- Add weather data to see how rain affects fares.
- Try tree-based models (XGBoost or Random Forest) for potential accuracy gains.
- Build a dashboard in Tableau or Power BI for more features.

7. Appendix

7.1 SQL Code

```
-----------------------------------------------------------------------
-- NYC Taxi Data Cleaning Script (SQLite)
-- Author: Rain999-sketch
-- Purpose: Clean 19.4M rows of NYC TLC trip records for analysis
-- Dataset: nyc_yellow_taxi_trip_records_from_Jan_to_Aug_2023.csv
-- Output: taxi_trips_final.csv (18,169,048 rows)
-----------------------------------------------------------------------

-- ========================================
-- STEP 1: IMPORT RAW CSV
-- ========================================
.mode csv
.import nyc_yellow_taxi_trip_records_from_Jan_to_Aug_2023.csv taxi_trips

-- ========================================
-- STEP 2: INITIAL INSPECTION
-- ========================================
-- Check total row count
SELECT COUNT(*) FROM taxi_trips;
-- Result: 19,493,059 rows

-- Preview the first 5 rows
SELECT * FROM taxi_trips LIMIT 5;
-- Result: first row was a header, later removed from the cleaned table

-- ========================================
-- STEP 3: DATA TYPE AUDIT
-- ========================================
-- Check declared schema (types may be blank due to SQLite dynamic typing)
PRAGMA table_info(taxi_trips);

-- Check actual storage class of values
SELECT typeof(VendorID), typeof(fare_amount) FROM taxi_trips LIMIT 1;
-- Result: both 'text'; confirms raw data is stored as text
```

```
-- ========================================
-- STEP 4: NULL VALUE CHECK
-- ========================================
-- Compare total rows with non-null counts per column
SELECT COUNT(*), COUNT(VendorID), COUNT(fare_amount) FROM taxi_trips;
-- Result: all counts equal 19,493,059; no NULLs found

-- ========================================
-- STEP 5: DUPLICATE CHECK
-- ========================================
-- Group by natural key (pickup, dropoff, fare, distance) to find
duplicates
SELECT tpep_pickup_datetime, tpep_dropoff_datetime, fare_amount,
trip_distance, COUNT(*)
FROM taxi_trips
GROUP BY tpep_pickup_datetime, tpep_dropoff_datetime, fare_amount,
trip_distance
HAVING COUNT(*) > 1
LIMIT 10;
-- Result: 1,016 duplicate groups totalling 2,032 rows
-- Note: We chose NOT to dedupe on this natural key because rows with
identical
-- keys but different VendorID or payment_type could be legitimately
distinct trips.
-- We used SELECT DISTINCT instead, which removes only rows identical
across all 20 columns.

-- ========================================
-- STEP 6: LOGICAL CONSISTENCY CHECK
-- ========================================
-- Find trips where dropoff is before or equal to pickup
SELECT COUNT(*) FROM taxi_trips WHERE tpep_dropoff_datetime <=
tpep_pickup_datetime;
-- Result: 7,434 impossible trips

-- ========================================
-- STEP 7: OUTLIER SCAN
-- ========================================
-- First pass: MIN/MAX directly on text columns returned incorrect results
-- because SQLite compares strings lexicographically ('999.0' >
'386983.63').
-- Corrected query casts to REAL / INTEGER before aggregation.
SELECT
    MAX(CAST(fare_amount AS REAL)) AS max_fare,
    MIN(CAST(fare_amount AS REAL)) AS min_fare,
    MAX(CAST(trip_distance AS REAL)) AS max_distance,
```

```
    MIN(CAST(trip_distance AS REAL)) AS min_distance,
    MAX(CAST(passenger_count AS INTEGER)) AS max_passengers,
    MIN(CAST(passenger_count AS INTEGER)) AS min_passengers
FROM taxi_trips;
-- Result: min fare = -959.9, max fare = 386,983.63,
-- max distance = 342,344.85 miles, max passengers = 9

-- ========================================
-- STEP 8: CATEGORICAL VALUE AUDIT
-- ========================================
-- Check distinct values in categorical columns
SELECT DISTINCT RatecodeID FROM taxi_trips;
-- Result: includes invalid 99.0 and empty ""

SELECT DISTINCT store_and_fwd_flag FROM taxi_trips;
-- Result: includes empty ""

SELECT DISTINCT payment_type FROM taxi_trips;
-- Result: valid (1, 2, 3, 4)

-- ========================================
-- STEP 9: BUILD THE FINAL CLEAN TABLE
-- ========================================
-- Filters applied:
-- fare_amount > 0 (remove refunds and errors)
-- trip_distance > 0 AND < 1000 (remove zero-distance and implausible
trips)
-- passenger_count between 1 and 6 (remove empty and oversized groups)
-- dropoff > pickup (remove negative-duration trips)
-- RatecodeID in valid range (1-6)
-- store_and_fwd_flag standardized to Y/N
-- Duplicates removed with SELECT DISTINCT

CREATE TABLE taxi_trips_final AS
SELECT DISTINCT
    CAST(VendorID AS INTEGER) AS VendorID,
    tpep_pickup_datetime,
    tpep_dropoff_datetime,
    CAST(passenger_count AS INTEGER) AS passenger_count,
    CAST(trip_distance AS REAL) AS trip_distance,
    CAST(RatecodeID AS INTEGER) AS RatecodeID,
    UPPER(TRIM(store_and_fwd_flag)) AS store_and_fwd_flag,
    CAST(PULocationID AS INTEGER) AS PULocationID,
    CAST(DOLocationID AS INTEGER) AS DOLocationID,
    CAST(payment_type AS INTEGER) AS payment_type,
    CAST(fare_amount AS REAL) AS fare_amount,
```

```
    CAST(extra AS REAL) AS extra,
    CAST(mta_tax AS REAL) AS mta_tax,
    CAST(tip_amount AS REAL) AS tip_amount,
    CAST(tolls_amount AS REAL) AS tolls_amount,
    CAST(improvement_surcharge AS REAL) AS improvement_surcharge,
    CAST(total_amount AS REAL) AS total_amount,
    CAST(congestion_surcharge AS REAL) AS congestion_surcharge,
    CAST(Airport_fee_20 AS REAL) AS Airport_fee_20,
    CAST(airport_fee_21 AS REAL) AS airport_fee_21
FROM taxi_trips
WHERE CAST(fare_amount AS REAL) > 0
  AND CAST(trip_distance AS REAL) > 0
  AND CAST(trip_distance AS REAL) < 1000
  AND CAST(passenger_count AS INTEGER) > 0
  AND CAST(passenger_count AS INTEGER) <= 6
  AND tpep_dropoff_datetime > tpep_pickup_datetime
  AND CAST(RatecodeID AS INTEGER) IN (1, 2, 3, 4, 5, 6)
  AND UPPER(TRIM(store_and_fwd_flag)) IN ('Y', 'N');

-- ========================================
-- STEP 10: VERIFY CLEANED TABLE
-- ========================================
-- Check final row count
SELECT COUNT(*) FROM taxi_trips_final;
-- Result: 18,169,048 rows (1,324,011 rows removed, 6.8% of original)

-- Preview the cleaned table
SELECT * FROM taxi_trips_final LIMIT 5;

-- Confirm schema and data types
PRAGMA table_info(taxi_trips_final);

SELECT typeof(VendorID), typeof(fare_amount) FROM taxi_trips_final LIMIT 1;
-- Result: 'integer' and 'real'; CAST() worked

-- ========================================
-- STEP 11: SAVE DATABASE AND EXPORT TO CSV
-- ========================================
-- Save the database for future queries
.save taxi_trips.db

-- Export the cleaned table to CSV for Python and Excel
.mode csv
.headers_on
.output taxi_trips_final.csv
SELECT * FROM taxi_trips_final;
```

```
.output stdout
```

7.2 Python Code

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sea
import warnings
warnings.filterwarnings('ignore')
from scipy import stats
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, r2_score

# Memory optimization
dtype_dict = {
    'VendorID': 'int8',
    'passenger_count': 'int8',
    'trip_distance': 'float32',
    'RatecodeID': 'int8',
    'PULocationID': 'int16',
    'DOLocationID': 'int16',
    'payment_type': 'int8',
    'fare_amount': 'float32',
    'extra': 'float32',
    'mta_tax': 'float32',
    'tip_amount': 'float32',
    'tolls_amount': 'float32',
    'improvement_surcharge': 'float32',
    'total_amount': 'float32',
    'congestion_surcharge': 'float32',
    'Airport_fee_20': 'float32',
    'airport_fee_21': 'float32'
}

# Loads the data
sw = pd.read_csv(
```

```
    'taxi_trips_final.csv',
    dtype=dtype_dict,
    parse_dates=['tpep_pickup_datetime', 'tpep_dropoff_datetime']
)

# Checking the data
print(f"Duplicate rows: {sw.duplicated().sum()}")
print("Rows loaded:", sw.shape[0])
print("Columns:", sw.shape[1])
print("Memory used:", round(sw.memory_usage(deep=True).sum() / 1e6, 2),
"MB")
print(sw.head())
print("\n---- DATA INFO ----")
sw.info()

print("\n---- DATA VALIDATION ----")

# Checking if the dropoff is always after pickup
bad_times = (sw['tpep_dropoff_datetime'] <=
sw['tpep_pickup_datetime']).sum()
print(f"Rows with dropoff <= pickup: {bad_times}")

# Checking airport fee columns
both_nonzero = ((sw['Airport_fee_20'] > 0) & (sw['airport_fee_21'] >
0)).sum()
print(f"Rows with both airport fees non-zero: {both_nonzero}")
print(f"Rows with Airport_fee_20 > 0: {(sw['Airport_fee_20'] > 0).sum()}")
print(f"Rows with airport_fee_21 > 0: {(sw['airport_fee_21'] > 0).sum()}")

# Checking unique values in categorical columns
for col in ['VendorID', 'RatecodeID', 'payment_type',
'store_and_fwd_flag']:
    print(f"{col}: {sw[col].unique()}")

# Comparing fare components sum to total_amount
sw['calc_total'] = (
    sw['fare_amount'] + sw['extra'] + sw['mta_tax'] +
    sw['tip_amount'] + sw['tolls_amount'] +
    sw['improvement_surcharge'] + sw['congestion_surcharge'] +
```

```
    sw['Airport_fee_20'] + sw['airport_fee_21']
)
mismatch = (sw['calc_total'] - sw['total_amount']).abs() > 0.01
print(f"Rows where components don't sum to total: {mismatch.sum()}
({mismatch.mean()*100:.2f}%)")

print(f"Fraction with non-zero congestion
surcharge:{(sw['congestion_surcharge'] != 0).mean():.4f}")
diff = sw['calc_total'] - sw['total_amount']
mismatched = sw[~np.isclose(sw['calc_total'], sw['total_amount'],
atol=0.01)]
matches_surcharge = np.isclose(
    diff[mismatched.index],
    sw.loc[mismatched.index, 'congestion_surcharge'],
    atol=0.01
)
print(f"Fraction of mismatched rows where gap = congestion surcharge:
{matches_surcharge.mean()*100:.2f}%")

sw['mismatch'] = ~np.isclose(sw['calc_total'], sw['total_amount'],
atol=0.01)
print("\nMismatch rate by vendor:")
print(sw.groupby('VendorID')['mismatch'].mean())

failing = sw[mismatch].head(3)
print(failing[['fare_amount', 'extra', 'mta_tax', 'tip_amount',
               'tolls_amount', 'improvement_surcharge',
'congestion_surcharge',
               'Airport_fee_20', 'airport_fee_21', 'calc_total',
'total_amount']])

print("\n---- FEATURE ENGINEERING ----")

# 1) Core trip metrics
sw['trip_duration_min'] = (sw['tpep_dropoff_datetime'] -
sw['tpep_pickup_datetime']).dt.total_seconds() / 60
sw['avg_speed_mph'] = sw['trip_distance'] / (sw['trip_duration_min'] / 60)
sw['fare_per_mile'] = sw['fare_amount'] / sw['trip_distance']
```

```
# 2) Time features
sw['pickup_hour'] = sw['tpep_pickup_datetime'].dt.hour
sw['pickup_day_of_week'] = sw['tpep_pickup_datetime'].dt.dayofweek  #
0=Mon, 6=Sun
sw['pickup_month'] = sw['tpep_pickup_datetime'].dt.month

# 3) Total surcharges
sw['total_surcharges'] = (
    sw['extra'] + sw['mta_tax'] + sw['tolls_amount'] +
    sw['improvement_surcharge'] + sw['congestion_surcharge'] +
    sw['Airport_fee_20'] + sw['airport_fee_21']
).fillna(0)

# Tip percentage
sw['tip_pct'] = sw['tip_amount'] / sw['fare_amount']

# Weekend flag
sw['is_weekend'] = (sw['pickup_day_of_week'] >= 5).astype('int8')

# 4) Handles division by 0
sw['avg_speed_mph'] = sw['avg_speed_mph'].replace([np.inf, -np.inf],
np.nan).fillna(0)
sw['fare_per_mile'] = sw['fare_per_mile'].replace([np.inf, -np.inf],
np.nan).fillna(0)
sw['tip_pct'] = sw['tip_pct'].replace([np.inf, -np.inf], np.nan).fillna(0)

# 5) Optimizing memory for the new columns
sw['total_surcharges'] = sw['total_surcharges'].astype('float32')
sw['tip_pct'] = sw['tip_pct'].astype('float32')

# 6. Final summary
print(f"New shape: {sw.shape[0]} rows, {sw.shape[1]} columns")
print("\nNew features added (12 total, including bins):")
print("  - distance_bin, fare_bin, duration_bin (for visualizations)")
print("  - trip_duration_min")
print("  - avg_speed_mph")
print("  - fare_per_mile")
print("  - pickup_hour, pickup_day_of_week, pickup_month")
print("  - total_surcharges")
```

```
print("  - tip_pct")
print("  - is_weekend")
print("\nSample of the new features:")
print(sw[['trip_duration_min', 'avg_speed_mph', 'fare_per_mile',
'total_surcharges', 'tip_pct', 'is_weekend']].head())

print("\n---- OUTLIER DETECTION ----")

# 1) Stats with percentiles
numeric_cols = [
    'trip_distance',
    'fare_amount',
    'trip_duration_min',
    'avg_speed_mph',
    'fare_per_mile',
    'total_surcharges',
    'tip_pct'
]

print(sw[numeric_cols].describe(percentiles=[0.01, 0.25, 0.5, 0.75,
0.99]).T)

# 2) Threshold counts
thresholds = {
    'trip_distance': 100,
    'fare_amount': 150,
    'trip_duration_min': 180,
    'avg_speed_mph': 80,
    'fare_per_mile': 20,
    'tip_pct': 0.5
}

print("\n---- OUTLIER COUNTS BY THRESHOLD ----")
for col, threshold in thresholds.items():
    n = (sw[col] > threshold).sum()
    pct = (n/len(sw))* 100
    print(f"{col}: {n} rows above {threshold} ({pct:.2f}%)")

print("\n---- CREATING CLEANED DATASET ----")
```

```
sw_eda = sw.sample(200000, random_state=42).copy()
sw_eda['trip_distance'] = sw_eda['trip_distance'].clip(upper=100)
sw_eda['fare_amount'] = sw_eda['fare_amount'].clip(upper=150)
sw_eda['trip_duration_min'] = sw_eda['trip_duration_min'].clip(upper=180)
sw_eda['avg_speed_mph'] = sw_eda['avg_speed_mph'].clip(upper=80)
sw_eda['fare_per_mile'] = sw_eda['fare_per_mile'].clip(upper=20)
sw_eda['tip_pct'] = sw_eda['tip_pct'].clip(upper=0.5)
print(f"Cleaned dataset shape: {sw_eda.shape}")

sw_eda_sample = sw_eda.sample(10000, random_state=42)
sw_eda_sample.to_csv('taxi_trips_eda_sample_small.csv', index=False)

print("\n---- BINNED DISTRIBUTIONS ----")

# Trip distance
distance_bins = [0, 1, 2, 3, 5, 10, 20, 30, 100]
distance_labels = ['0-1', '1-2', '2-3', '3-5', '5-10', '10-20', '20-30',
'30+']
sw_eda['distance_bin'] = pd.cut(sw_eda['trip_distance'],
bins=distance_bins, labels=distance_labels)

# Fare amount
fare_bins = [0, 5, 10, 15, 20, 30, 50, 150]
fare_labels = ['0-5', '5-10', '10-15', '15-20', '20-30', '30-50', '50+']
sw_eda['fare_bin'] = pd.cut(sw_eda['fare_amount'], bins=fare_bins,
labels=fare_labels)

# Trip duration
duration_bins = [0, 5, 10, 15, 20, 30, 60, 180]
duration_labels = ['0-5', '5-10', '10-15', '15-20', '20-30', '30-60',
'60+']
sw_eda['duration_bin'] = pd.cut(sw_eda['trip_duration_min'],
bins=duration_bins, labels=duration_labels)

# 1) Trip distance distribution
plt.figure(figsize=(10, 6))
sw_eda['distance_bin'].value_counts().sort_index().plot(kind='bar',
color='steelblue', edgecolor='black')
plt.title('Trip Distance Distribution (Binned)')
```

```
plt.xlabel('Distance (miles)')
plt.ylabel('Number of Trips')
plt.xticks(rotation=45)
plt.grid(axis='y', alpha=0.3)
plt.tight_layout()
plt.show()
plt.close()

# 2) Fare amount distribution
plt.figure(figsize=(10, 6))
sw_eda['fare_bin'].value_counts().sort_index().plot(kind='bar',
color='coral', edgecolor='black')
plt.title('Fare Amount Distribution (Binned)')
plt.xlabel('Fare ($)')
plt.ylabel('Number of Trips')
plt.xticks(rotation=45)
plt.grid(axis='y', alpha=0.3)
plt.tight_layout()
plt.show()
plt.close()

# 3) Trip duration distribution
plt.figure(figsize=(10, 6))
sw_eda['duration_bin'].value_counts().sort_index().plot(kind='bar',
color='mediumseagreen', edgecolor='black')
plt.title('Trip Duration Distribution (Binned)')
plt.xlabel('Duration (minutes)')
plt.ylabel('Number of Trips')
plt.xticks(rotation=45)
plt.grid(axis='y', alpha=0.3)
plt.tight_layout()
plt.show()
plt.close()

print("\n---- CLEAN EDA ----")
# Time aggregations
avg_fare_hour =
sw_eda.groupby('pickup_hour')['fare_amount'].mean().reset_index()
plt.figure(figsize=(10, 6))
```

```
sea.lineplot(data=avg_fare_hour, x='pickup_hour', y='fare_amount',
marker='o')
plt.title('Average Fare by Hour of Day')
plt.xlabel('Hour')
plt.ylabel('Avg Fare ($)')
plt.grid(True)
plt.show()
plt.close()

avg_fare_day =
sw_eda.groupby('pickup_day_of_week')['fare_amount'].mean().reset_index()
plt.figure(figsize=(10, 6))
sea.barplot(data=avg_fare_day, x='pickup_day_of_week', y='fare_amount')
plt.title('Average Fare by Day of Week (0=Mon, 6=Sun)')
plt.xlabel('Day')
plt.ylabel('Avg Fare ($)')
plt.show()
plt.close()

weekend_compare = sw_eda.groupby('is_weekend')['fare_amount'].agg(['mean',
'count']).reset_index()
weekend_compare.columns = ['is_weekend', 'avg_fare', 'trip_count']
print("\nWeekend vs Weekday Comparison:")
print(weekend_compare)

print("\nClean EDA complete.")

print("\n---- 5 AM AIRPORT HYPOTHESIS TEST ----")

# Method 1) Check RatecodeID distribution at 5 AM vs noon
five_am = sw[sw['pickup_hour'] == 5]
noon = sw[sw['pickup_hour'] == 12]

print(f"5 AM trips: {len(five_am):,}")
print("\nRatecodeID distribution at 5 AM (%):")
print((five_am['RatecodeID'].value_counts(normalize=True) * 100).round(2))

print("\nRatecodeID distribution at 12 PM (%):")
print((noon['RatecodeID'].value_counts(normalize=True) * 100).round(2))
```

```
# Method 2) Check airport zones at 5 AM
airport_zones = [1, 132, 138]  # Newark, JFK, LaGuardia
airport_at_5am = five_am[five_am['PULocationID'].isin(airport_zones)]
airport_at_noon = noon[noon['PULocationID'].isin(airport_zones)]

print(f"\nAirport pickups at 5 AM: {len(airport_at_5am):,}
({len(airport_at_5am)/len(five_am)*100:.2f}%)")
print(f"Airport pickups at 12 PM: {len(airport_at_noon):,}
({len(airport_at_noon)/len(noon)*100:.2f}%)")

# Average fare from airport vs non-airport at 5 AM
five_am_airport_fare = airport_at_5am['fare_amount'].mean()
five_am_non_airport_fare =
five_am[~five_am['PULocationID'].isin(airport_zones)]['fare_amount'].mean()
print(f"\n5 AM avg fare, Airport pickups: ${five_am_airport_fare:.2f}")
print(f"5 AM avg fare, Non-airport pickups:
${five_am_non_airport_fare:.2f}")

# Is the 5 AM spike airport-driven, or do non-airport trips also spike?
sw['fare_c'] = sw['fare_amount'].clip(upper=150).astype('float64')
sw['is_airport_trip'] = (
    sw['PULocationID'].isin(airport_zones) |
    sw['DOLocationID'].isin(airport_zones) |
    sw['RatecodeID'].isin([2, 3])
)

by_hour = (
    sw.groupby(['pickup_hour', 'is_airport_trip'])['fare_c']
      .mean()
      .unstack()
)
by_hour.columns = ['non_airport', 'airport']
by_hour['all_trips'] = sw.groupby('pickup_hour')['fare_c'].mean()

print("\nAverage fare by hour (airport vs non-airport):")
print(by_hour.round(2))

five_am_share = sw[sw['pickup_hour'] == 5]['is_airport_trip'].mean() * 100
```

```
noon_share = sw[sw['pickup_hour'] == 12]['is_airport_trip'].mean() * 100

print(f"\nAirport-related share at 5 AM: {five_am_share:.2f}%")
print(f"Airport-related share at 12 PM: {noon_share:.2f}%")

print("\n---- CORRELATION HEATMAP ----")

# 1) Selecting numbers to test correlation
corr_cols = [
    'fare_amount',
    'trip_distance',
    'passenger_count',
    'trip_duration_min',
    'avg_speed_mph',
    'fare_per_mile',
    'total_surcharges',
    'tip_pct',
    'pickup_hour',
    'pickup_day_of_week',
    'is_weekend'
]

# 2) Calculating the correlation
corr_matrix = sw_eda[corr_cols].corr()

# 3) Heatmap
plt.figure(figsize=(12, 10))
sea.heatmap(corr_matrix, annot=True, cmap='coolwarm', fmt='.2f',
linewidths=0.5)
plt.title('Correlation Matrix')
plt.show()
plt.close()

print("Correlation heatmap displayed.")

print("\n---- STATISTICAL TESTING ----")

weekday_fares = sw_eda[sw_eda['is_weekend'] == 0]['fare_amount']
weekend_fares = sw_eda[sw_eda['is_weekend'] == 1]['fare_amount']
```

```python
# Welch's t-test
t_stat, p_val = stats.ttest_ind(weekday_fares, weekend_fares,
equal_var=False)
print(f"T-statistic: {t_stat:.2f}")
print(f"P-value: {p_val:.4f}")

# Confidence interval for the difference in means
mean_diff = weekend_fares.mean() - weekday_fares.mean()
se = np.sqrt(
    weekend_fares.var() / len(weekend_fares) +
    weekday_fares.var() / len(weekday_fares)
)
ci_low = mean_diff - 1.96 * se
ci_high = mean_diff + 1.96 * se
print(f"Mean difference (weekend - weekday): ${mean_diff:.2f}")
print(f"95% CI: [${ci_low:.2f}, ${ci_high:.2f}]")

if p_val < 0.05:
    print("Result: Statistically significant difference (p < 0.05)")
else:
    print("Result: No statistically significant difference (p >= 0.05)")

print("\n---- LINEAR REGRESSION MODEL ----")

airport_zones = [1, 132, 138]
sw_eda['is_airport_trip'] = (
    sw_eda['PULocationID'].isin(airport_zones) |
    sw_eda['DOLocationID'].isin(airport_zones) |
    sw_eda['RatecodeID'].isin([2, 3])
).astype(int)

# 1) features chosen

features = ['trip_distance', 'trip_duration_min', 'passenger_count',
'is_weekend', 'is_airport_trip']
X = sw_eda[features]
y = sw_eda['fare_amount']
```

```python
# 2) Splitting the data between training and testing data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

# 3) Training the model
model = LinearRegression()
model.fit(X_train, y_train)

# 4) Making predictions
y_pred = model.predict(X_test)

# 5) Evaluating the model
mae = mean_absolute_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print(f"Mean Absolute Error: ${mae:.2f}")
print(f"R-squared: {r2:.4f}")

# Baseline comparison
X_base = sw_eda[['trip_distance']]
X_train_b, X_test_b, y_train_b, y_test_b = train_test_split(
    X_base, y, test_size=0.2, random_state=42
)
baseline = LinearRegression()
baseline.fit(X_train_b, y_train_b)
y_pred_b = baseline.predict(X_test_b)

r2_base = r2_score(y_test_b, y_pred_b)
mae_base = mean_absolute_error(y_test_b, y_pred_b)

print(f"\nBaseline (distance only):")
print(f"  R-squared: {r2_base:.4f}")
print(f"  MAE: ${mae_base:.2f}")
print(f"\nFull model improvement over baseline:")
print(f"  R² gain: {r2 - r2_base:.4f}")

# 6) Coefficients
print("\nModel Coefficients:")
for feature, coef in zip(features, model.coef_):
```

```
    print(f"  {feature}: {coef:.4f}")
print(f"  Intercept: {model.intercept_:.2f}")

print("\n---- RESIDUAL PLOT ----")
plt.figure(figsize=(8, 6))
sea.residplot(x=y_test, y=y_pred, lowess=True, line_kws={'color': 'red',
'lw': 1})
plt.xlabel('Actual Fare')
plt.ylabel('Residuals')
plt.title('Residual Plot')
plt.show()
plt.close()

sw_eda.to_csv('taxi_trips_eda_sample.csv', index=False)
print("Cleaned sample saved as 'taxi_trips_eda_sample.csv'")

print("\n---- CATEGORICAL AGGREGATIONS ----")

avg_fare_payment =
sw_eda.groupby('payment_type')['fare_amount'].mean().reset_index()
avg_fare_payment.columns = ['payment_type', 'avg_fare']
print("Average Fare by Payment Type:")
print(avg_fare_payment)

avg_fare_vendor = sw_eda.groupby('VendorID')['fare_amount'].agg(['mean',
'count']).reset_index()
avg_fare_vendor.columns = ['VendorID', 'avg_fare', 'trip_count']
print("\nAverage Fare by Vendor:")
print(avg_fare_vendor)

avg_tip_payment =
sw_eda.groupby('payment_type')['tip_pct'].mean().reset_index()
avg_tip_payment.columns = ['payment_type', 'avg_tip_pct']
print("\nAverage Tip Percentage by Payment Type:")
print(avg_tip_payment)

sw_eda['fare_per_min'] = sw_eda['fare_amount'] /
sw_eda['trip_duration_min']
```

```
valid = sw_eda['trip_duration_min'] > 0
rev_by_distance = (
    sw_eda[valid]
    .groupby('distance_bin')['fare_per_min']
    .median()
    .round(2)
)
print("\nMedian revenue per occupied minute by distance bin:")
print(rev_by_distance)

print("\n---- ACTUAL VS PREDICTED PLOT ----")
plt.figure(figsize=(8, 6))
sea.scatterplot(x=y_test, y=y_pred, alpha=0.3)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--')
plt.xlabel('Actual Fare')
plt.ylabel('Predicted Fare')
plt.title('Actual vs Predicted Fare')
plt.show()
plt.close()

print("\n---- EXPORT TO EXCEL ----")

print("\n---- PREPARING EXCEL EXPORT ----")

# 1) Summary dataframe
summary_sw = pd.DataFrame({
    'Metric': [
        'Total Trips (Sample)',
        'Average Fare ($)',
        'Average Distance (miles)',
        'Average Duration (min)',
        'Weekday Avg Fare ($)',
        'Weekend Avg Fare ($)',
        'Model R-squared',
        'Model MAE ($)'
    ],
    'Value': [
        len(sw_eda),
        sw_eda['fare_amount'].mean(),
```

```
        sw_eda['trip_distance'].mean(),
        sw_eda['trip_duration_min'].mean(),
        weekday_fares.mean(),
        weekend_fares.mean(),
        r2,
        mae
    ]
})

# 2) Coefficents dataframe
coef_sw = pd.DataFrame({
    'Feature': features + ['Intercept'],
    'Coefficient': list(model.coef_) + [model.intercept_]
})

with pd.ExcelWriter('taxi_analysis_report.xlsx', engine='xlsxwriter') as
writer:
    summary_sw.to_excel(writer, sheet_name='Summary', index=False)
    coef_sw.to_excel(writer, sheet_name='Model Coefficients', index=False)
    weekend_compare.to_excel(writer, sheet_name='Weekend vs Weekday',
index=False)

    avg_fare_payment.to_excel(writer, sheet_name='Avg Fare by Payment',
index=False)
    avg_fare_vendor.to_excel(writer, sheet_name='Avg Fare by Vendor',
index=False)
    avg_tip_payment.to_excel(writer, sheet_name='Avg Tip by Payment',
index=False)

    corr_matrix.to_excel(writer, sheet_name='Correlation Matrix')

print("Excel report saved as 'taxi_analysis_report.xlsx'")
print("\n---- SCRIPT COMPLETE ----")
```